CHAPTER 6

Scipy : high-level scientific computing

Authors: *Gaël Varoquaux, Adrien Chauve, Andre Espaze, Emmanuelle Gouillart, Ralf Gommers*

Scipy

The `scipy` package contains various toolboxes dedicated to common issues in scientific computing. Its different submodules correspond to different applications, such as interpolation, integration, optimization, image processing, statistics, special functions, etc.

Tip: `scipy` can be compared to other standard scientific-computing libraries, such as the GSL (GNU Scientific Library for C and C++), or Matlab's toolboxes. `scipy` is the core package for scientific routines in Python; it is meant to operate efficiently on `numpy` arrays, so that numpy and scipy work hand in hand.

Before implementing a routine, it is worth checking if the desired data processing is not already implemented in Scipy. As non-professional programmers, scientists often tend to **re-invent the wheel**, which leads to buggy, non-optimal, difficult-to-share and unmaintainable code. By contrast, `Scipy`'s routines are optimized and tested, and should therefore be used when possible.

Chapters contents

- *File input/output:* `scipy.io`
- *Special functions:* `scipy.special`
- *Linear algebra operations:* `scipy.linalg`
- *Interpolation:* `scipy.interpolate`

- *Optimization and fit:* `scipy.optimize`
- *Statistics and random numbers:* `scipy.stats`
- *Numerical integration:* `scipy.integrate`
- *Fast Fourier transforms:* `scipy.fftpack`
- *Signal processing:* `scipy.signal`
- *Image manipulation:* `scipy.ndimage`
- *Summary exercises on scientific computing*
- *Full code examples for the scipy chapter*

Warning: This tutorial is far from an introduction to numerical computing. As enumerating the different submodules and functions in scipy would be very boring, we concentrate instead on a few examples to give a general idea of how to use `scipy` for scientific computing.

`scipy` is composed of task-specific sub-modules:

`scipy.cluster`	Vector quantization / Kmeans
`scipy.constants`	Physical and mathematical constants
`scipy.fftpack`	Fourier transform
`scipy.integrate`	Integration routines
`scipy.interpolate`	Interpolation
`scipy.io`	Data input and output
`scipy.linalg`	Linear algebra routines
`scipy.ndimage`	n-dimensional image package
`scipy.odr`	Orthogonal distance regression
`scipy.optimize`	Optimization
`scipy.signal`	Signal processing
`scipy.sparse`	Sparse matrices
`scipy.spatial`	Spatial data structures and algorithms
`scipy.special`	Any special mathematical functions
`scipy.stats`	Statistics

Tip: They all depend on `numpy`, but are mostly independent of each other. The standard way of importing Numpy and these Scipy modules is:

```
>>> import numpy as np
>>> from scipy import stats  # same for other sub-modules
```

The main `scipy` namespace mostly contains functions that are really numpy functions (try `scipy.cos is np.cos`). Those are exposed for historical reasons; there's no reason to use `import scipy` in your code.

6.1 File input/output: `scipy.io`

Matlab files: Loading and saving:

```
>>> from scipy import io as spio
>>> a = np.ones((3, 3))
>>> spio.savemat('file.mat', {'a': a}) # savemat expects a dictionary
```

(continues on next page)

(continued from previous page)

```
>>> data = spio.loadmat('file.mat')
>>> data['a']
array([[1.,  1.,  1.],
       [1.,  1.,  1.],
       [1.,  1.,  1.]])
```

Warning: Python / Matlab mismatches, *eg* matlab does not represent 1D arrays

```
>>> a = np.ones(3)
>>> a
array([1.,  1.,  1.])
>>> spio.savemat('file.mat', {'a': a})
>>> spio.loadmat('file.mat')['a']
array([[1.,  1.,  1.]])
```

Notice the difference?

Image files: Reading images:

```
>>> import imageio
>>> imageio.imread('fname.png')
Array(...)
>>> # Matplotlib also has a similar function
>>> import matplotlib.pyplot as plt
>>> plt.imread('fname.png')
array(...)
```

See also:

- Load text files: `numpy.loadtxt()`/`numpy.savetxt()`
- Clever loading of text/csv files: `numpy.genfromtxt()`/`numpy.recfromcsv()`
- Fast and efficient, but numpy-specific, binary format: `numpy.save()`/`numpy.load()`
- More advanced input/output of images in scikit-image: `skimage.io`

6.2 Special functions: `scipy.special`

Special functions are transcendental functions. The docstring of the `scipy.special` module is well-written, so we won't list all functions here. Frequently used ones are:

- Bessel function, such as `scipy.special.jn()` (nth integer order Bessel function)
- Elliptic function (`scipy.special.ellipj()` for the Jacobian elliptic function, ...)
- Gamma function: `scipy.special.gamma()`, also note `scipy.special.gammaln()` which will give the log of Gamma to a higher numerical precision.
- Erf, the area under a Gaussian curve: `scipy.special.erf()`

6.3 Linear algebra operations: `scipy.linalg`

Tip: The `scipy.linalg` module provides standard linear algebra operations, relying on an underlying efficient implementation (BLAS, LAPACK).

- The `scipy.linalg.det()` function computes the determinant of a square matrix:

```
>>> from scipy import linalg
>>> arr = np.array([[1, 2],
...                 [3, 4]])
>>> linalg.det(arr)
-2.0
>>> arr = np.array([[3, 2],
...                 [6, 4]])
>>> linalg.det(arr)
0.0
>>> linalg.det(np.ones((3, 4)))
Traceback (most recent call last):
...
ValueError: expected square matrix
```

- The `scipy.linalg.inv()` function computes the inverse of a square matrix:

```
>>> arr = np.array([[1, 2],
...                 [3, 4]])
>>> iarr = linalg.inv(arr)
>>> iarr
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
>>> np.allclose(np.dot(arr, iarr), np.eye(2))
True
```

  Finally computing the inverse of a singular matrix (its determinant is zero) will raise `LinAlgError`:

```
>>> arr = np.array([[3, 2],
...                 [6, 4]])
>>> linalg.inv(arr)
Traceback (most recent call last):
...
...LinAlgError: singular matrix
```

- More advanced operations are available, for example singular-value decomposition (SVD):

```
>>> arr = np.arange(9).reshape((3, 3)) + np.diag([1, 0, 1])
>>> uarr, spec, vharr = linalg.svd(arr)
```

  The resulting array spectrum is:

```
>>> spec
array([14.88982544,  0.45294236,  0.29654967])
```

  The original matrix can be re-composed by matrix multiplication of the outputs of `svd` with `np.dot`:

```
>>> sarr = np.diag(spec)
>>> svd_mat = uarr.dot(sarr).dot(vharr)
>>> np.allclose(svd_mat, arr)
True
```

  SVD is commonly used in statistics and signal processing. Many other standard decompositions (QR, LU, Cholesky, Schur), as well as solvers for linear systems, are available in `scipy.linalg`.

6.4 Interpolation: `scipy.interpolate`

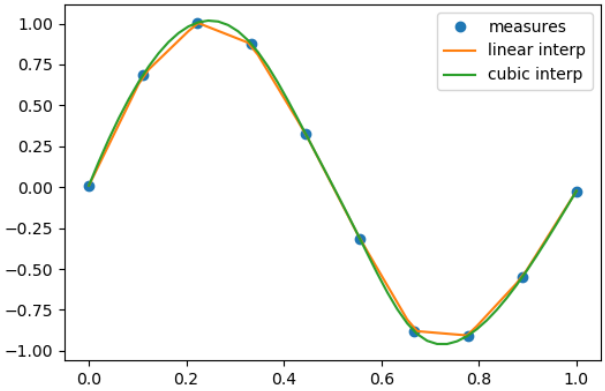
`scipy.interpolate` is useful for fitting a function from experimental data and thus evaluating points where no measure exists. The module is based on the FITPACK Fortran subroutines.

By imagining experimental data close to a sine function:

```
>>> measured_time = np.linspace(0, 1, 10)
>>> noise = (np.random.random(10)*2 - 1) * 1e-1
>>> measures = np.sin(2 * np.pi * measured_time) + noise
```

`scipy.interpolate.interp1d` can build a linear interpolation function:

```
>>> from scipy.interpolate import interp1d
>>> linear_interp = interp1d(measured_time, measures)
```



Then the result can be evaluated at the time of interest:

```
>>> interpolation_time = np.linspace(0, 1, 50)
>>> linear_results = linear_interp(interpolation_time)
```

A cubic interpolation can also be selected by providing the `kind` optional keyword argument:

```
>>> cubic_interp = interp1d(measured_time, measures, kind='cubic')
>>> cubic_results = cubic_interp(interpolation_time)
```

`scipy.interpolate.interp2d` is similar to `scipy.interpolate.interp1d`, but for 2-D arrays. Note that for the *interp* family, the interpolation points must stay within the range of given data points. See the summary exercise on *Maximum wind speed prediction at the Sprogø station* for a more advanced spline interpolation example.

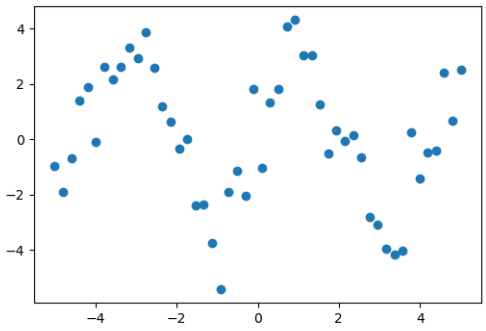
6.5 Optimization and fit: `scipy.optimize`

Optimization is the problem of finding a numerical solution to a minimization or equality.

Tip: The `scipy.optimize` module provides algorithms for function minimization (scalar or multi-dimensional), curve fitting and root finding.

```
>>> from scipy import optimize
```

6.5.1 Curve fitting

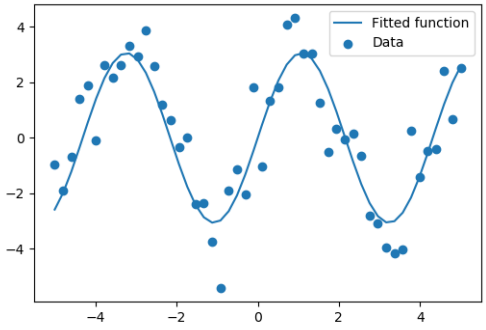


Suppose we have data on a sine wave, with some noise:

```
>>> x_data = np.linspace(-5, 5, num=50)
>>> y_data = 2.9 * np.sin(1.5 * x_data) + np.random.normal(size=50)
```

If we know that the data lies on a sine wave, but not the amplitudes or the period, we can find those by least squares curve fitting. First we have to define the test function to fit, here a sine with unknown amplitude and period:

```
>>> def test_func(x, a, b):
...     return a * np.sin(b * x)
```



We then use `scipy.optimize.curve_fit()` to find a and b:

```
>>> params, params_covariance = optimize.curve_fit(test_func, x_data, y_data, p0=[2, 2])
>>> print(params)
[3.05931973  1.45754553]
```

Exercise: Curve fitting of temperature data

The temperature extremes in Alaska for each month, starting in January, are given by (in degrees Celcius):

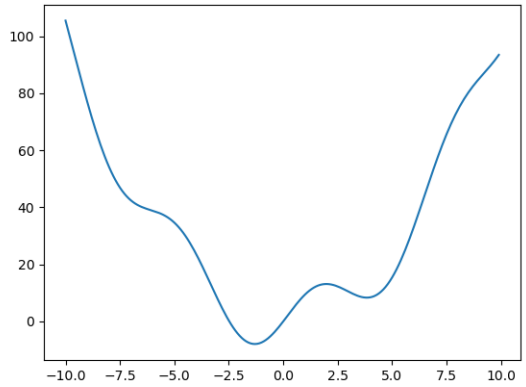
```
max:  17,  19,  21,  28,  33,  38, 37,  37,  31,  23,  19,  18
min: -62, -59, -56, -46, -32, -18, -9, -13, -25, -46, -52, -58
```

1. Plot these temperature extremes.
2. Define a function that can describe min and max temperatures. Hint: this function has to have a period of 1 year. Hint: include a time offset.
3. Fit this function to the data with `scipy.optimize.curve_fit()`.
4. Plot the result. Is the fit reasonable? If not, why?

5. Is the time offset for min and max temperatures the same within the fit accuracy?

solution

6.5.2 Finding the minimum of a scalar function



Let's define the following function:

```
>>> def f(x):
...     return x**2 + 10*np.sin(x)
```

and plot it:

```
>>> x = np.arange(-10, 10, 0.1)
>>> plt.plot(x, f(x))
>>> plt.show()
```

This function has a global minimum around -1.3 and a local minimum around 3.8.

Searching for minimum can be done with `scipy.optimize.minimize()`, given a starting point x0, it returns the location of the minimum that it has found:

result type

The result of `scipy.optimize.minimize()` is a compound object comprising all information on the convergence

```
>>> result = optimize.minimize(f, x0=0)
>>> result
      fun: -7.9458233756...
 hess_inv: array([[0.0858...]])
      jac: array([-1.19209...e-06])
  message: 'Optimization terminated successfully.'
     nfev: 18
      nit: 5
     njev: 6
   status: 0
  success: True
        x: array([-1.30644...])
>>> result.x # The coordinate of the minimum
array([-1.30644...])
```

Methods: As the function is a smooth function, gradient-descent based methods are good options. The lBFGS algorithm is a good choice in general:

```
>>> optimize.minimize(f, x0=0, method="L-BFGS-B")
      fun: array([-7.94582338])
 hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
      jac: array([-1.42108547e-06])
  message: ...'CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL'
     nfev: 12
      nit: 5
   status: 0
  success: True
        x: array([-1.30644013])
```

Note how it cost only 12 functions evaluation above to find a good value for the minimum.

Global minimum: A possible issue with this approach is that, if the function has local minima, the algorithm may find these local minima instead of the global minimum depending on the initial point x0:

```
>>> res = optimize.minimize(f, x0=3, method="L-BFGS-B")
>>> res.x
array([3.83746709])
```

If we don't know the neighborhood of the global minimum to choose the initial point, we need to resort to costlier global optimization. To find the global minimum, we use `scipy.optimize.basinhopping()` (added in version 0.12.0 of Scipy). It combines a local optimizer with sampling of starting points:

```
>>> optimize.basinhopping(f, 0)
                    nfev: 1725
 minimization_failures: 0
                     fun: -7.9458233756152845
                       x: array([-1.30644001])
                 message: ['requested number of basinhopping iterations completed successfully']
                    njev: 575
                     nit: 100
```

Note: `scipy` used to contain the routine *anneal*, it has been removed in SciPy 0.16.0.

Constraints: We can constrain the variable to the interval `(0, 10)` using the "bounds" argument:

A list of bounds

As `minimize()` works in general with x multidimensionsal, the "bounds" argument is a list of bound on each dimension.

```
>>> res = optimize.minimize(f, x0=1,
...                         bounds=((0, 10), ))
>>> res.x
array([0.])
```

Tip: What has happened? Why are we finding 0, which is not a mimimum of our function.

Minimizing functions of several variables

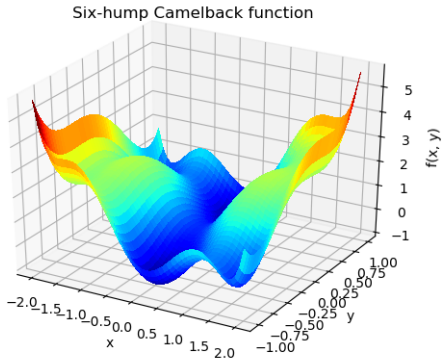
To minimize over several variables, the trick is to turn them into a function of a multi-dimensional variable (a vector). See for instance the exercise on 2D minimization below.

Note: `scipy.optimize.minimize_scalar()` is a function with dedicated methods to minimize functions of only one variable.

See also:

Finding minima of function is discussed in more details in the advanced chapter: *Mathematical optimization: finding minima of functions*.

Exercise: 2-D minimization



The six-hump camelback function

$$f(x, y) = (4 - 2.1x^2 + \frac{x^4}{3})x^2 + xy + (4y^2 - 4)y^2$$

has multiple global and local minima. Find the global minima of this function.

Hints:

- Variables can be restricted to $-2 < x < 2$ and $-1 < y < 1$.
- Use `numpy.meshgrid()` and `matplotlib.pyplot.imshow()` to find visually the regions.
- Use `scipy.optimize.minimize()`, optionally trying out several of its methods.

How many global minima are there, and what is the function value at those points? What happens for an initial guess of $(x, y) = (0, 0)$?

solution

6.5.3 Finding the roots of a scalar function

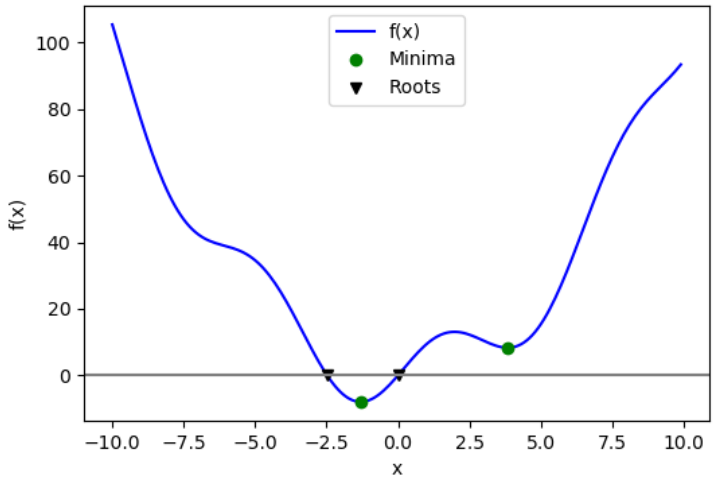
To find a root, i.e. a point where $f(x) = 0$, of the function f above we can use `scipy.optimize.root()`:

```
>>> root = optimize.root(f, x0=1)  # our initial guess is 1
>>> root    # The full result
    fjac: array([[-1.]])
     fun: array([0.])
 message: 'The solution converged.'
    nfev: 10
     qtf: array([1.33310463e-32])
       r: array([-10.])
  status: 1
 success: True
       x: array([0.])
>>> root.x  # Only the root found
array([0.])
```

Note that only one root is found. Inspecting the plot of f reveals that there is a second root around -2.5. We find the exact value of it by adjusting our initial guess:

```
>>> root2 = optimize.root(f, x0=-2.5)
>>> root2.x
array([-2.47948183])
```

Note: `scipy.optimize.root()` also comes with a variety of algorithms, set via the "method" argument.



Now that we have found the minima and roots of `f` and used curve fitting on it, we put all those results together in a single plot:

See also:

You can find all algorithms and functions with similar functionalities in the documentation of `scipy.optimize`.

See the summary exercise on *Non linear least squares curve fitting: application to point extraction in topographical lidar data* for another, more advanced example.

6.6 Statistics and random numbers: `scipy.stats`

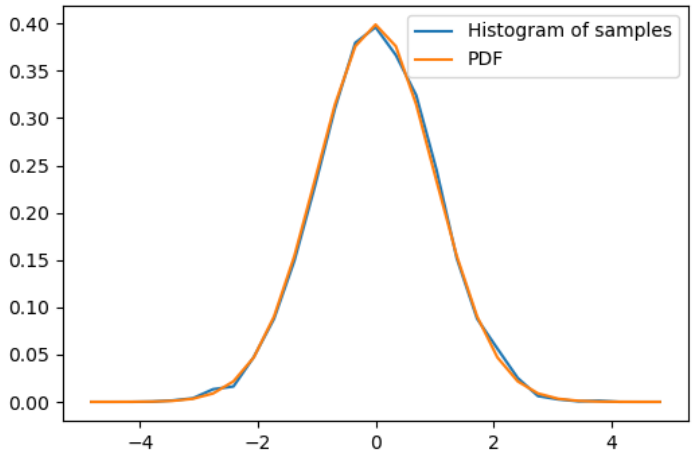
The module `scipy.stats` contains statistical tools and probabilistic descriptions of random processes. Random number generators for various random process can be found in `numpy.random`.

6.6.1 Distributions: histogram and probability density function

Given observations of a random process, their histogram is an estimator of the random process's PDF (probability density function):

```
>>> samples = np.random.normal(size=1000)
>>> bins = np.arange(-4, 5)
>>> bins
array([-4, -3, -2, -1,  0,  1,  2,  3,  4])
>>> histogram = np.histogram(samples, bins=bins, density=True)[0]
>>> bins = 0.5*(bins[1:] + bins[:-1])
>>> bins
array([-3.5, -2.5, -1.5, -0.5,  0.5,  1.5,  2.5,  3.5])
>>> from scipy import stats
>>> pdf = stats.norm.pdf(bins)  # norm is a distribution object

>>> plt.plot(bins, histogram)
[<matplotlib.lines.Line2D object at ...>]
>>> plt.plot(bins, pdf)
[<matplotlib.lines.Line2D object at ...>]
```



The distribution objects

`scipy.stats.norm` is a distribution object: each distribution in `scipy.stats` is represented as an object. Here it's the normal distribution, and it comes with a PDF, a CDF, and much more.

If we know that the random process belongs to a given family of random processes, such as normal processes, we can do a maximum-likelihood fit of the observations to estimate the parameters of the underlying distribution. Here we fit a normal process to the observed data:

```
>>> loc, std = stats.norm.fit(samples)
>>> loc
-0.045256707...
>>> std
0.9870331586...
```

Exercise: Probability distributions

Generate 1000 random variates from a gamma distribution with a shape parameter of 1, then plot a histogram from those samples. Can you plot the pdf on top (it should match)?

Extra: the distributions have many useful methods. Explore them by reading the docstring or by using tab completion. Can you recover the shape parameter 1 by using the `fit` method on your random variates?

6.6.2 Mean, median and percentiles

The mean is an estimator of the center of the distribution:

```
>>> np.mean(samples)
-0.0452567074...
```

The median another estimator of the center. It is the value with half of the observations below, and half above:

```
>>> np.median(samples)
-0.0580280347...
```

Tip: Unlike the mean, the median is not sensitive to the tails of the distribution. It is "robust".

Exercise: Compare mean and median on samples of a Gamma distribution

Which one seems to be the best estimator of the center for the Gamma distribution?

The median is also the percentile 50, because 50% of the observation are below it:

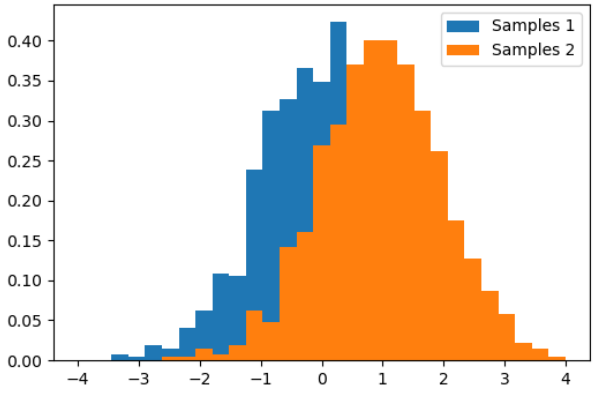
```
>>> stats.scoreatpercentile(samples, 50)
-0.0580280347...
```

Similarly, we can calculate the percentile 90:

```
>>> stats.scoreatpercentile(samples, 90)
1.2315935511...
```

Tip: The percentile is an estimator of the CDF: cumulative distribution function.

6.6.3 Statistical tests



A statistical test is a decision indicator. For instance, if we have two sets of observations, that we assume are generated from Gaussian processes, we can use a T-test to decide whether the means of two sets of observations are significantly different:

```
>>> a = np.random.normal(0, 1, size=100)
>>> b = np.random.normal(1, 1, size=10)
>>> stats.ttest_ind(a, b)
(array(-3.177574054...), 0.0019370639...)
```

Tip: The resulting output is composed of:

- The T statistic value: it is a number the sign of which is proportional to the difference between the two random processes and the magnitude is related to the significance of this difference.
- the *p value*: the probability of both processes being identical. If it is close to 1, the two process are almost certainly identical. The closer it is to zero, the more likely it is that the processes have different means.

See also:

The chapter on *statistics* introduces much more elaborate tools for statistical testing and statistical data loading and visualization outside of scipy.

6.7 Numerical integration: `scipy.integrate`

6.7.1 Function integrals

The most generic integration routine is `scipy.integrate.quad()`. To compute $\int_0^{\pi/2} sin(t)dt$:

```
>>> from scipy.integrate import quad
>>> res, err = quad(np.sin, 0, np.pi/2)
>>> np.allclose(res, 1)   # res is the result, is should be close to 1
True
>>> np.allclose(err, 1 - res)  # err is an estimate of the err
True
```

Other integration schemes are available: `scipy.integrate.fixed_quad()`, `scipy.integrate.quadrature()`, `scipy.integrate.romberg()`...

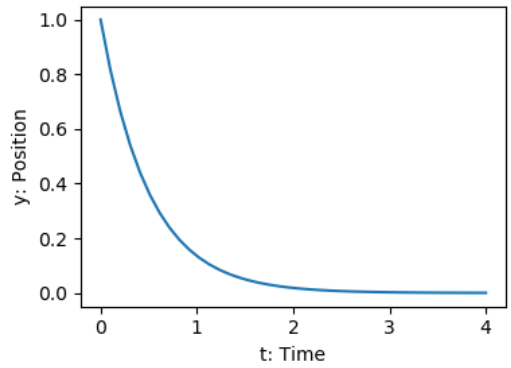
6.7.2 Integrating differential equations

`scipy.integrate` also features routines for integrating Ordinary Differential Equations (ODE). In particular, `scipy.integrate.odeint()` solves ODE of the form:

```
dy/dt = rhs(y1, y2, .., t0,...)
```

As an introduction, let us solve the ODE $\frac{dy}{dt} = -2y$ between $t = 0...4$, with the initial condition $y(t = 0) = 1$. First the function computing the derivative of the position needs to be defined:

```
>>> def calc_derivative(ypos, time):
...     return -2 * ypos
```



Then, to compute y as a function of time:

```
>>> from scipy.integrate import odeint
>>> time_vec = np.linspace(0, 4, 40)
>>> y = odeint(calc_derivative, y0=1, t=time_vec)
```

Let us integrate a more complex ODE: a damped spring-mass oscillator. The position of a mass attached to a spring obeys the 2nd order *ODE* $y'' + 2\varepsilon\omega_0 y' + \omega_0^2 y = 0$ with $\omega_0^2 = k/m$ with k the spring constant, m the mass and $\varepsilon = c/(2m\omega_0)$ with c the damping coefficient. We set:

```
>>> mass = 0.5  # kg
>>> kspring = 4  # N/m
>>> cviscous = 0.4  # N s/m
```

Hence:

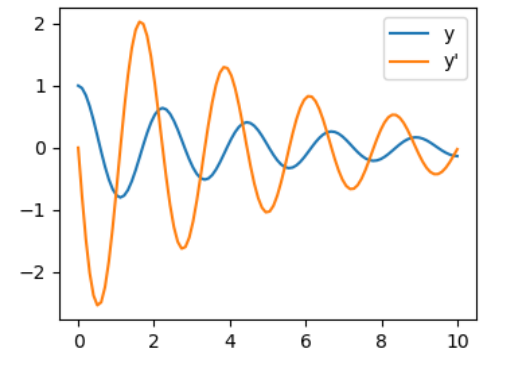
```
>>> eps = cviscous / (2 * mass * np.sqrt(kspring/mass))
>>> omega = np.sqrt(kspring / mass)
```

The system is underdamped, as:

```
>>> eps < 1
True
```

For `odeint()`, the 2nd order equation needs to be transformed in a system of two first-order equations for the vector $Y = (y, y')$: the function computes the velocity and acceleration:

```
>>> def calc_deri(yvec, time, eps, omega):
...     return (yvec[1], -2.0 * eps * omega * yvec[1] - omega **2 * yvec[0])
```



Integration of the system follows:

```
>>> time_vec = np.linspace(0, 10, 100)
>>> yinit = (1, 0)
>>> yarr = odeint(calc_deri, yinit, time_vec, args=(eps, omega))
```

Tip: `scipy.integrate.odeint()` uses the LSODA (Livermore Solver for Ordinary Differential equations with Automatic method switching for stiff and non-stiff problems), see the ODEPACK Fortran library for more details.

See also:

Partial Differental Equations

There is no Partial Differential Equations (PDE) solver in Scipy. Some Python packages for solving PDE's are available, such as fipy or SfePy.

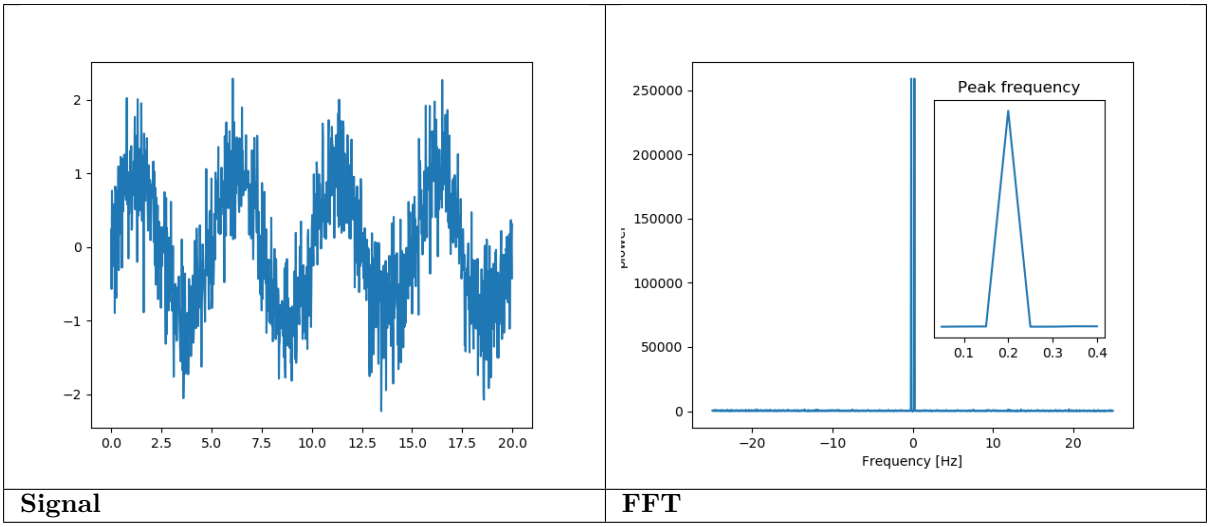
6.8 Fast Fourier transforms: `scipy.fftpack`

The `scipy.fftpack` module computes fast Fourier transforms (FFTs) and offers utilities to handle them. The main functions are:

- `scipy.fftpack.fft()` to compute the FFT
- `scipy.fftpack.fftfreq()` to generate the sampling frequencies
- `scipy.fftpack.ifft()` computes the inverse FFT, from frequency space to signal space

As an illustration, a (noisy) input signal (`sig`), and its FFT:

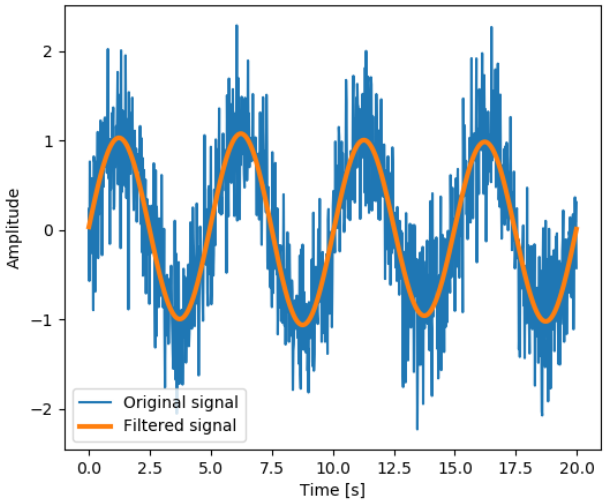
```
>>> from scipy import fftpack
>>> sig_fft = fftpack.fft(sig)
>>> freqs = fftpack.fftfreq(sig.size, d=time_step)
```



Signal | **FFT**

As the signal comes from a real function, the Fourier transform is symmetric.

The peak signal frequency can be found with `freqs[power.argmax()]`



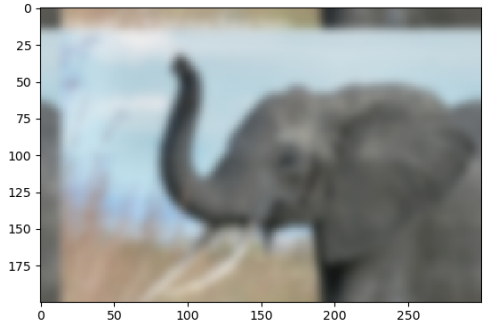
Setting the Fourrier component above this frequency to zero and inverting the FFT with `scipy.fftpack.ifft()`, gives a filtered signal.

Note: The code of this example can be found *here*

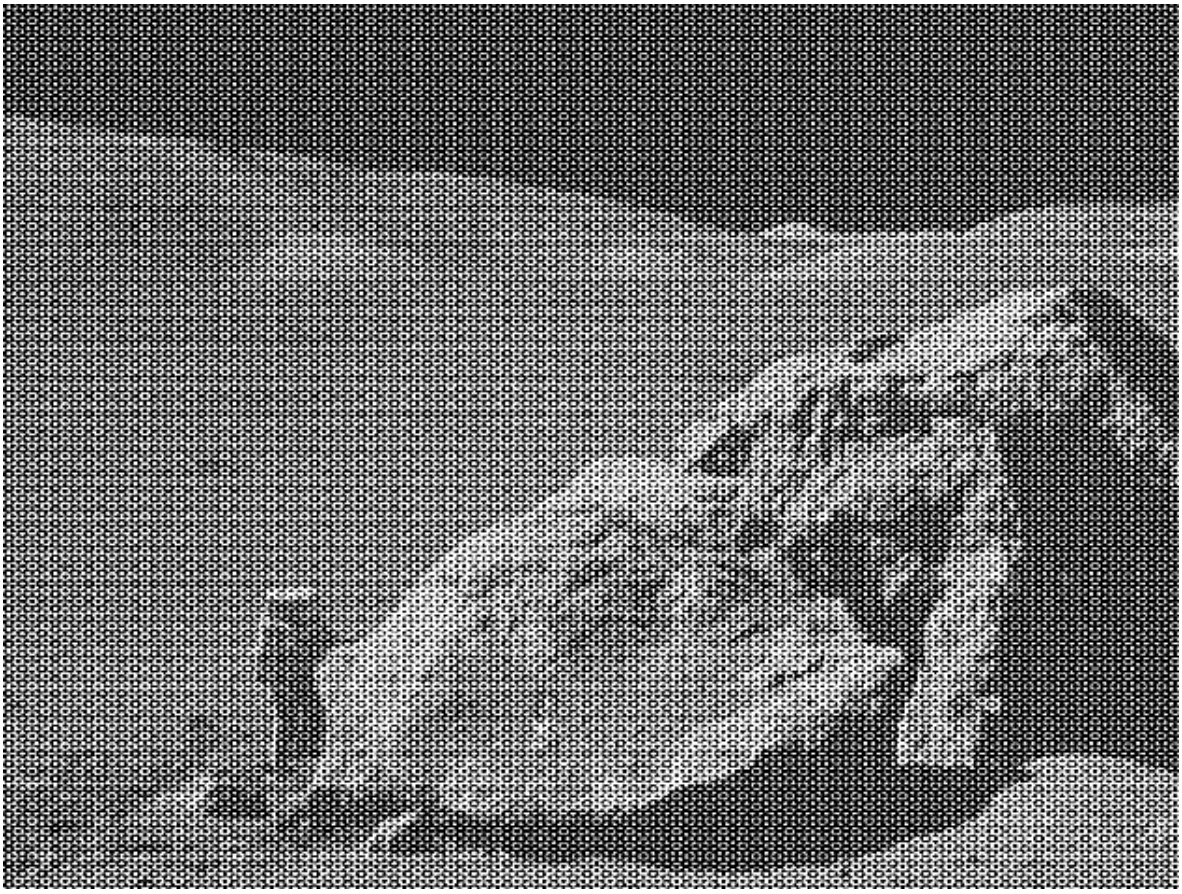
numpy.fft

Numpy also has an implementation of FFT (`numpy.fft`). However, the scipy one should be preferred, as it uses more efficient underlying implementations.

Fully worked examples:

Crude periodicity finding (*link*)	Gaussian image blur (*link*)
	

Exercise: Denoise moon landing image

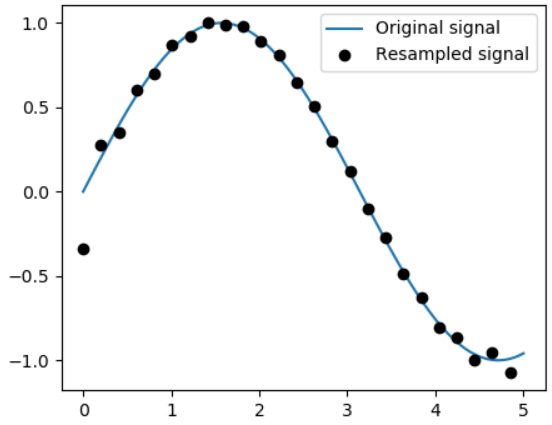


1. Examine the provided image `moonlanding.png`, which is heavily contaminated with periodic noise. In this exercise, we aim to clean up the noise using the Fast Fourier Transform.
2. Load the image using `matplotlib.pyplot.imread()`.
3. Find and use the 2-D FFT function in `scipy.fftpack`, and plot the spectrum (Fourier transform of) the image. Do you have any trouble visualising the spectrum? If so, why?
4. The spectrum consists of high and low frequency components. The noise is contained in the high-frequency part of the spectrum, so set some of those components to zero (use array slicing).
5. Apply the inverse Fourier transform to see the resulting image.

Solution

6.9 Signal processing: `scipy.signal`

Tip: `scipy.signal` is for typical signal processing: 1D, regularly-sampled signals.



Resampling `scipy.signal.resample()`: resample a signal to n points using FFT.

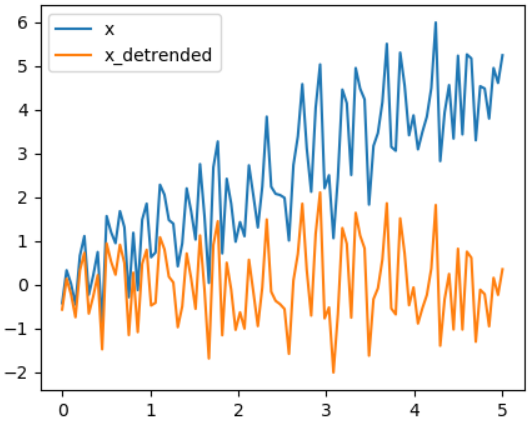
```
>>> t = np.linspace(0, 5, 100)
>>> x = np.sin(t)

>>> from scipy import signal
>>> x_resampled = signal.resample(x, 25)

>>> plt.plot(t, x)
[<matplotlib.lines.Line2D object at ...>]
>>> plt.plot(t[::4], x_resampled, 'ko')
[<matplotlib.lines.Line2D object at ...>]
```

Tip: Notice how on the side of the window the resampling is less accurate and has a rippling effect.

This resampling is different from the *interpolation* provided by `scipy.interpolate` as it only applies to regularly sampled data.



Detrending `scipy.signal.detrend()`: remove linear trend from signal:

```
>>> t = np.linspace(0, 5, 100)
>>> x = t + np.random.normal(size=100)

>>> from scipy import signal
>>> x_detrended = signal.detrend(x)

>>> plt.plot(t, x)
```

(continues on next page)

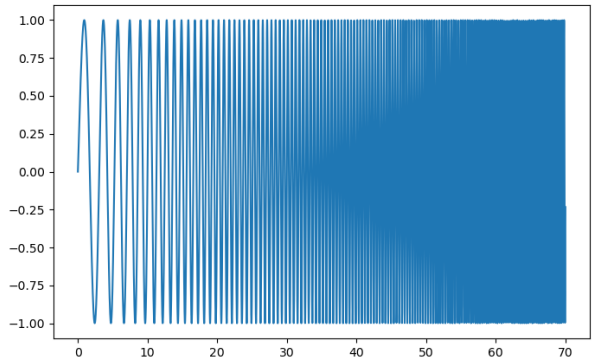
(continued from previous page)

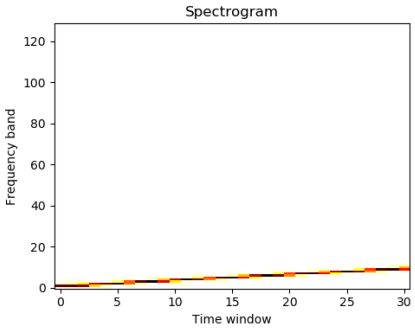
```
[<matplotlib.lines.Line2D object at ...>]
>>> plt.plot(t, x_detrended)
[<matplotlib.lines.Line2D object at ...>]
```

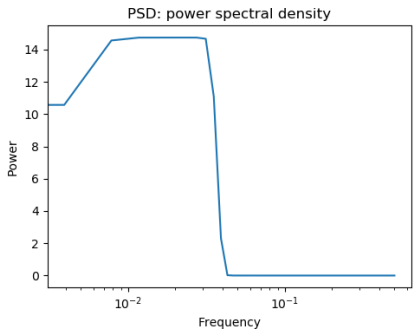
Filtering: For non-linear filtering, `scipy.signal` has filtering (median filter `scipy.signal.medfilt()`, Wiener `scipy.signal.wiener()`), but we will discuss this in the image section.

Tip: `scipy.signal` also has a full-blown set of tools for the design of linear filter (finite and infinite response filters), but this is out of the scope of this tutorial.

Spectral analysis: `scipy.signal.spectrogram()` compute a spectrogram –frequency spectrums over consecutive time windows–, while `scipy.signal.welch()` comptes a power spectrum density (PSD).







6.10 Image manipulation: `scipy.ndimage`

`scipy.ndimage` provides manipulation of n-dimensional arrays as images.

6.10.1 Geometrical transformations on images

Changing orientation, resolution, ..

```
>>> from scipy import misc  # Load an image
>>> face = misc.face(gray=True)

>>> from scipy import ndimage # Shift, roate and zoom it
>>> shifted_face = ndimage.shift(face, (50, 50))
>>> shifted_face2 = ndimage.shift(face, (50, 50), mode='nearest')
>>> rotated_face = ndimage.rotate(face, 30)
>>> cropped_face = face[50:-50, 50:-50]
>>> zoomed_face = ndimage.zoom(face, 2)
```

(continues on next page)

(continued from previous page)

```
>>> zoomed_face.shape
(1536, 2048)
```



```
>>> plt.subplot(151)
<matplotlib.axes._subplots.AxesSubplot object at 0x...>

>>> plt.imshow(shifted_face, cmap=plt.cm.gray)
<matplotlib.image.AxesImage object at 0x...>

>>> plt.axis('off')
(-0.5, 1023.5, 767.5, -0.5)

>>> # etc.
```

6.10.2 Image filtering

Generate a noisy face:

```
>>> from scipy import misc
>>> face = misc.face(gray=True)
>>> face = face[:512, -512:]  # crop out square on right
>>> import numpy as np
>>> noisy_face = np.copy(face).astype(np.float)
>>> noisy_face += face.std() * 0.5 * np.random.standard_normal(face.shape)
```

Apply a variety of filters on it:

```
>>> blurred_face = ndimage.gaussian_filter(noisy_face, sigma=3)
>>> median_face = ndimage.median_filter(noisy_face, size=5)
>>> from scipy import signal
>>> wiener_face = signal.wiener(noisy_face, (5, 5))
```



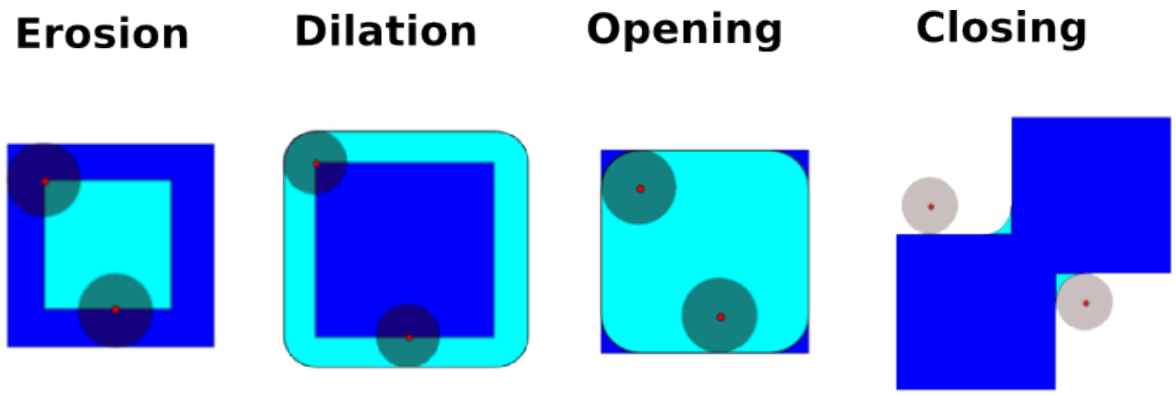
Other filters in `scipy.ndimage.filters` and `scipy.signal` can be applied to images.

Exercise

Compare histograms for the different filtered images.

6.10.3 Mathematical morphology

Tip: Mathematical morphology stems from set theory. It characterizes and transforms geometrical structures. Binary (black and white) images, in particular, can be transformed using this theory: the sets to be transformed are the sets of neighboring non-zero-valued pixels. The theory was also extended to gray-valued images.



Mathematical-morphology operations use a *structuring element* in order to modify geometrical structures.

Let us first generate a structuring element:

```
>>> el = ndimage.generate_binary_structure(2, 1)
>>> el
array([[False, True, False],
       [...True, True, True],
       [False, True, False]])
>>> el.astype(np.int)
array([[0, 1, 0],
       [1, 1, 1],
       [0, 1, 0]])
```

- **Erosion** `scipy.ndimage.binary_erosion()`

```
>>> a = np.zeros((7, 7), dtype=np.int)
>>> a[1:6, 2:5] = 1
>>> a
array([[0, 0, 0, 0, 0, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 0, 0, 0, 0, 0]])
>>> ndimage.binary_erosion(a).astype(a.dtype)
array([[0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0]])
>>> #Erosion removes objects smaller than the structure
>>> ndimage.binary_erosion(a, structure=np.ones((5,5))).astype(a.dtype)
array([[0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
```

(continues on next page)

(continued from previous page)

```
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0]])
```

- **Dilation** `scipy.ndimage.binary_dilation()`

```
>>> a = np.zeros((5, 5))
>>> a[2, 2] = 1
>>> a
array([[0.,  0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.,  0.],
       [0.,  0.,  1.,  0.,  0.],
       [0.,  0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.,  0.]])
>>> ndimage.binary_dilation(a).astype(a.dtype)
array([[0.,  0.,  0.,  0.,  0.],
       [0.,  0.,  1.,  0.,  0.],
       [0.,  1.,  1.,  1.,  0.],
       [0.,  0.,  1.,  0.,  0.],
       [0.,  0.,  0.,  0.,  0.]])
```

- **Opening** `scipy.ndimage.binary_opening()`

```
>>> a = np.zeros((5, 5), dtype=np.int)
>>> a[1:4, 1:4] = 1
>>> a[4, 4] = 1
>>> a
array([[0, 0, 0, 0, 0],
       [0, 1, 1, 1, 0],
       [0, 1, 1, 1, 0],
       [0, 1, 1, 1, 0],
       [0, 0, 0, 0, 1]])
>>> # Opening removes small objects
>>> ndimage.binary_opening(a, structure=np.ones((3, 3))).astype(np.int)
array([[0, 0, 0, 0, 0],
       [0, 1, 1, 1, 0],
       [0, 1, 1, 1, 0],
       [0, 1, 1, 1, 0],
       [0, 0, 0, 0, 0]])
>>> # Opening can also smooth corners
>>> ndimage.binary_opening(a).astype(np.int)
array([[0, 0, 0, 0, 0],
       [0, 0, 1, 0, 0],
       [0, 1, 1, 1, 0],
       [0, 0, 1, 0, 0],
       [0, 0, 0, 0, 0]])
```

- **Closing:** `scipy.ndimage.binary_closing()`

Exercise

Check that opening amounts to eroding, then dilating.

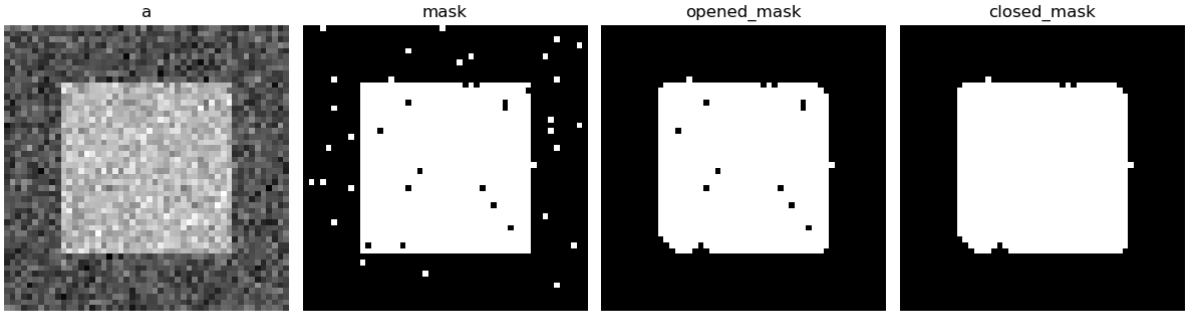
An opening operation removes small structures, while a closing operation fills small holes. Such operations can therefore be used to "clean" an image.

```
>>> a = np.zeros((50, 50))
>>> a[10:-10, 10:-10] = 1
>>> a += 0.25 * np.random.standard_normal(a.shape)
```

(continues on next page)

(continued from previous page)

```
>>> mask = a>=0.5
>>> opened_mask = ndimage.binary_opening(mask)
>>> closed_mask = ndimage.binary_closing(opened_mask)
```



Exercise

Check that the area of the reconstructed square is smaller than the area of the initial square. (The opposite would occur if the closing step was performed *before* the opening).

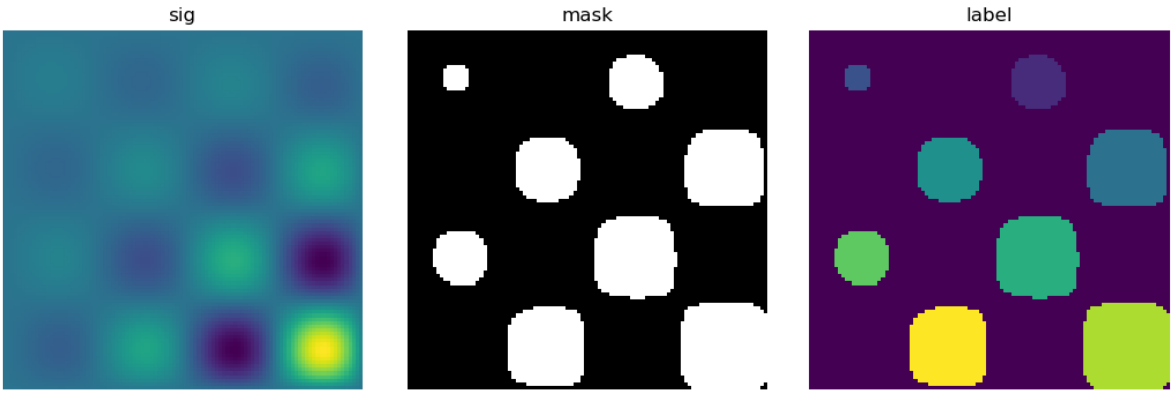
For *gray-valued* images, eroding (resp. dilating) amounts to replacing a pixel by the minimal (resp. maximal) value among pixels covered by the structuring element centered on the pixel of interest.

```
>>> a = np.zeros((7, 7), dtype=np.int)
>>> a[1:6, 1:6] = 3
>>> a[4, 4] = 2; a[2, 3] = 1
>>> a
array([[0, 0, 0, 0, 0, 0, 0],
       [0, 3, 3, 3, 3, 3, 0],
       [0, 3, 3, 1, 3, 3, 0],
       [0, 3, 3, 3, 3, 3, 0],
       [0, 3, 3, 3, 2, 3, 0],
       [0, 3, 3, 3, 3, 3, 0],
       [0, 0, 0, 0, 0, 0, 0]])
>>> ndimage.grey_erosion(a, size=(3, 3))
array([[0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 1, 1, 0, 0],
       [0, 0, 3, 2, 2, 0, 0],
       [0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0]])
```

6.10.4 Connected components and measurements on images

Let us first generate a nice synthetic binary image.

```
>>> x, y = np.indices((100, 100))
>>> sig = np.sin(2*np.pi*x/50.) * np.sin(2*np.pi*y/50.) * (1+x*y/50.**2)**2
>>> mask = sig > 1
```

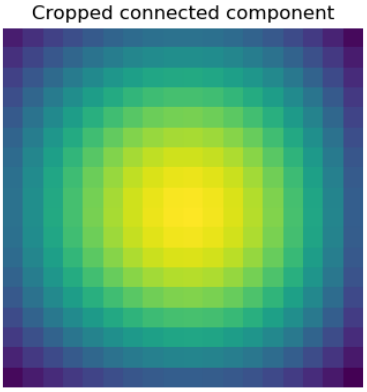


`scipy.ndimage.label()` assigns a different label to each connected component:

```
>>> labels, nb = ndimage.label(mask)
>>> nb
8
```

Now compute measurements on each connected component:

```
>>> areas = ndimage.sum(mask, labels, range(1, labels.max()+1))
>>> areas   # The number of pixels in each connected component
array([190.,   45.,  424.,  278.,  459.,  190.,  549.,  424.])
>>> maxima = ndimage.maximum(sig, labels, range(1, labels.max()+1))
>>> maxima  # The maximum signal in each connected component
array([ 1.80238238,  1.13527605,  5.51954079,  2.49611818, 6.71673619,
        1.80238238, 16.76547217,  5.51954079])
```



Extract the 4th connected component, and crop the array around it:

```
>>> ndimage.find_objects(labels==4)
[(slice(30L, 48L, None), slice(30L, 48L, None))]
>>> sl = ndimage.find_objects(labels==4)
>>> from matplotlib import pyplot as plt
>>> plt.imshow(sig[sl[0]])
<matplotlib.image.AxesImage object at ...>
```

See the summary exercise on *Image processing application: counting bubbles and unmolten grains* for a more advanced example.

6.11 Summary exercises on scientific computing

The summary exercises use mainly Numpy, Scipy and Matplotlib. They provide some real-life examples of scientific computing with Python. Now that the basics of working with Numpy and Scipy have been introduced, the interested user is invited to try these exercises.

6.11.1 Maximum wind speed prediction at the Sprogø station

The exercise goal is to predict the maximum wind speed occurring every 50 years even if no measure exists for such a period. The available data are only measured over 21 years at the Sprogø meteorological station located in Denmark. First, the statistical steps will be given and then illustrated with functions from the scipy.interpolate module. At the end the interested readers are invited to compute results from raw data and in a slightly different approach.

Statistical approach

The annual maxima are supposed to fit a normal probability density function. However such function is not going to be estimated because it gives a probability from a wind speed maxima. Finding the maximum wind speed occurring every 50 years requires the opposite approach, the result needs to be found from a defined probability. That is the quantile function role and the exercise goal will be to find it. In the current model, it is supposed that the maximum wind speed occurring every 50 years is defined as the upper 2% quantile.

By definition, the quantile function is the inverse of the cumulative distribution function. The latter describes the probability distribution of an annual maxima. In the exercise, the cumulative probability `p_i` for a given year `i` is defined as `p_i = i/(N+1)` with `N = 21`, the number of measured years. Thus it will be possible to calculate the cumulative probability of every measured wind speed maxima. From those experimental points, the scipy.interpolate module will be very useful for fitting the quantile function. Finally the 50 years maxima is going to be evaluated from the cumulative probability of the 2% quantile.

Computing the cumulative probabilities

The annual wind speeds maxima have already been computed and saved in the numpy format in the file `examples/max-speeds.npy`, thus they will be loaded by using numpy:

```
>>> import numpy as np
>>> max_speeds = np.load('intro/summary-exercises/examples/max-speeds.npy')
>>> years_nb = max_speeds.shape[0]
```

Following the cumulative probability definition `p_i` from the previous section, the corresponding values will be:

```
>>> cprob = (np.arange(years_nb, dtype=np.float32) + 1)/(years_nb + 1)
```

and they are assumed to fit the given wind speeds:

```
>>> sorted_max_speeds = np.sort(max_speeds)
```

Prediction with UnivariateSpline

In this section the quantile function will be estimated by using the `UnivariateSpline` class which can represent a spline from points. The default behavior is to build a spline of degree 3 and points can have different weights according to their reliability. Variants are `InterpolatedUnivariateSpline` and `LSQUnivariateSpline` on which errors checking is going to change. In case a 2D spline is wanted, the `BivariateSpline` class family is provided. All those classes for 1D and 2D splines use the FITPACK Fortran subroutines, that's why a lower library access is available through the `splrep` and `splev` functions for respectively representing and evaluating a spline. Moreover interpolation functions without the use of FITPACK parameters are also provided for simpler use (see `interp1d`, `interp2d`, `barycentric_interpolate` and so on).

For the Sprogø maxima wind speeds, the `UnivariateSpline` will be used because a spline of degree 3 seems to correctly fit the data:

```
>>> from scipy.interpolate import UnivariateSpline
>>> quantile_func = UnivariateSpline(cprob, sorted_max_speeds)
```

The quantile function is now going to be evaluated from the full range of probabilities:

```
>>> nprob = np.linspace(0, 1, 1e2)
>>> fitted_max_speeds = quantile_func(nprob)
```

In the current model, the maximum wind speed occurring every 50 years is defined as the upper 2% quantile. As a result, the cumulative probability value will be:

```
>>> fifty_prob = 1. - 0.02
```

So the storm wind speed occurring every 50 years can be guessed by:

```
>>> fifty_wind = quantile_func(fifty_prob)
>>> fifty_wind
array(32.97989825...)
```

The results are now gathered on a Matplotlib figure:

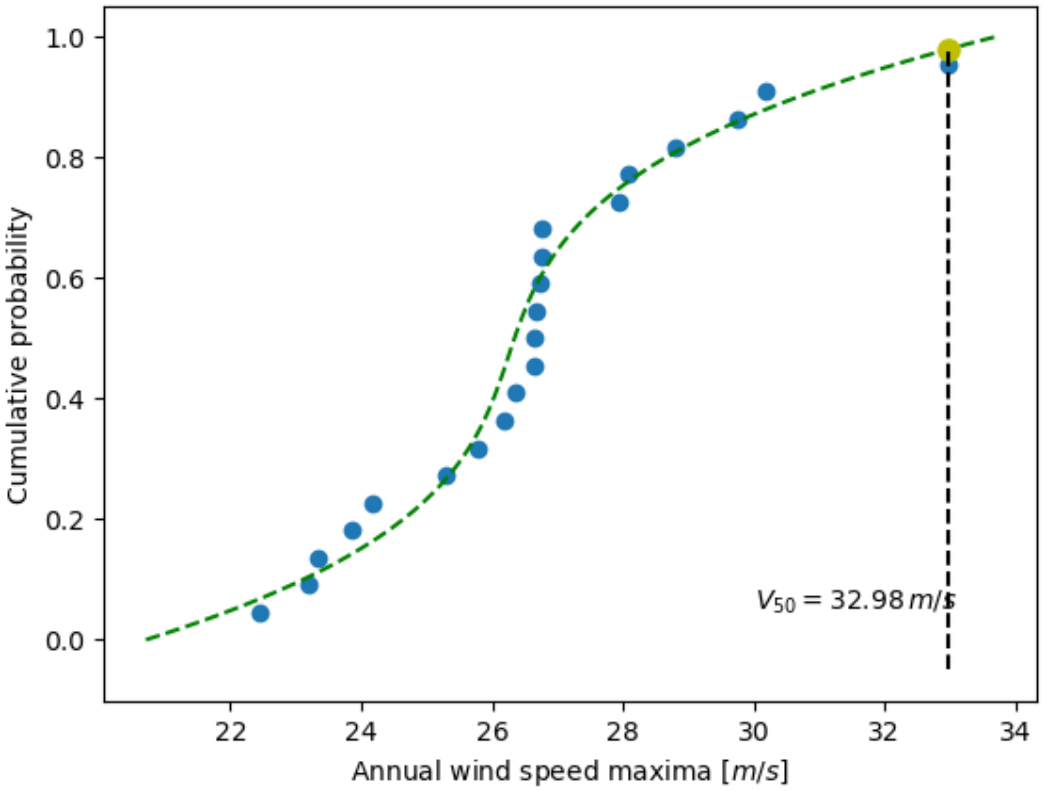


Fig. 1: Solution: Python source file

Exercise with the Gumbell distribution

The interested readers are now invited to make an exercise by using the wind speeds measured over 21 years. The measurement period is around 90 minutes (the original period was around 10 minutes but the file size has been reduced for making the exercise setup easier). The data are stored in numpy format inside the file `examples/sprog-windspeeds.npy`. Do not look at the source code for the plots until you have completed the exercise.

- The first step will be to find the annual maxima by using numpy and plot them as a matplotlib bar figure.

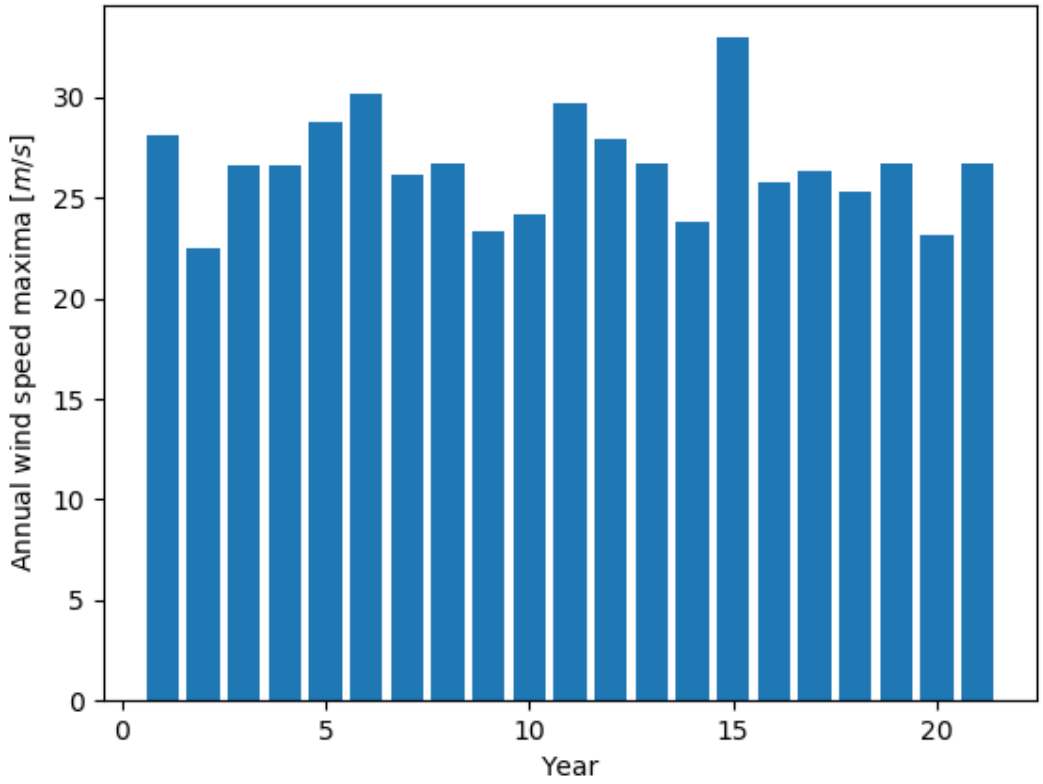


Fig. 2: Solution: Python source file

- The second step will be to use the Gumbell distribution on cumulative probabilities `p_i` defined as `-log( -log(p_i) )` for fitting a linear quantile function (remember that you can define the degree of the `UnivariateSpline`). Plotting the annual maxima versus the Gumbell distribution should give you the following figure.

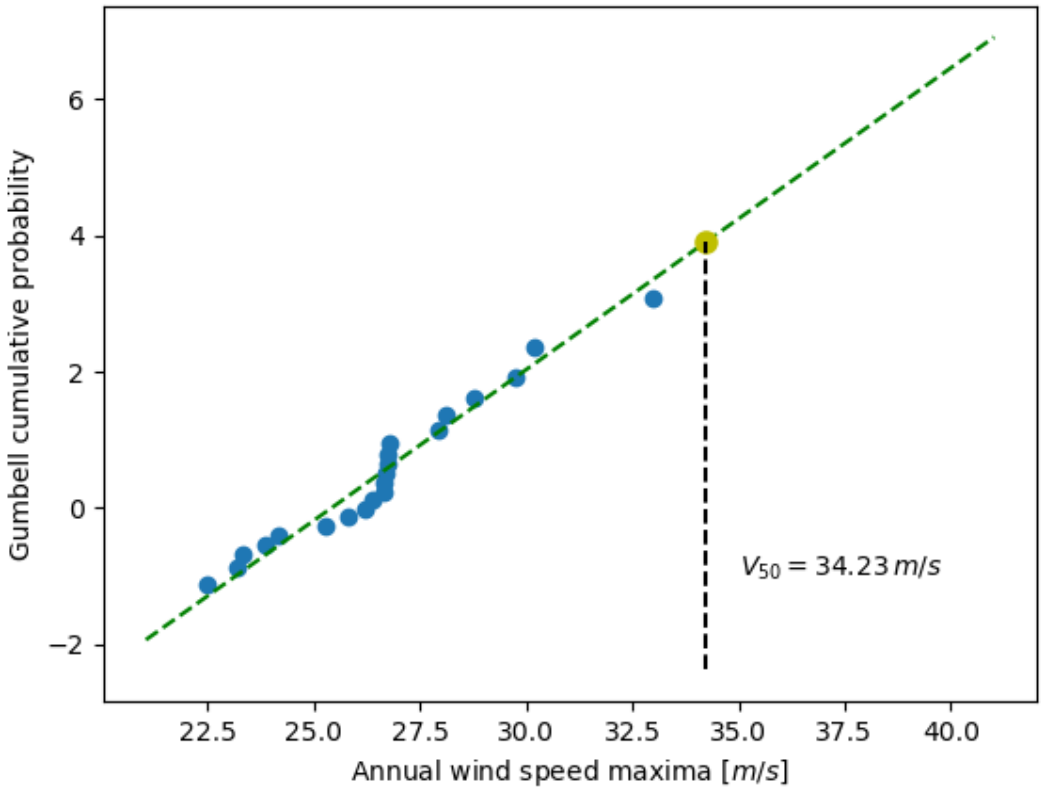


Fig. 3: Solution: Python source file

- The last step will be to find 34.23 m/s for the maximum wind speed occurring every 50 years.

6.11.2 Non linear least squares curve fitting: application to point extraction in topographical lidar data

The goal of this exercise is to fit a model to some data. The data used in this tutorial are lidar data and are described in details in the following introductory paragraph. If you're impatient and want to practice now, please skip it and go directly to *Loading and visualization*.

Introduction

Lidars systems are optical rangefinders that analyze property of scattered light to measure distances. Most of them emit a short light impulsion towards a target and record the reflected signal. This signal is then processed to extract the distance between the lidar system and the target.

Topographical lidar systems are such systems embedded in airborne platforms. They measure distances between the platform and the Earth, so as to deliver information on the Earth's topography (see[1] for more details).

In this tutorial, the goal is to analyze the waveform recorded by the lidar system[2]. Such a signal

[1] Mallet, C. and Bretar, F. Full-Waveform Topographic Lidar: State-of-the-Art. *ISPRS Journal of Photogrammetry and Remote Sensing* 64(1), pp.1-16, January 2009 http://dx.doi.org/10.1016/j.isprsjprs.2008.09.007

[2] The data used for this tutorial are part of the demonstration data available for the FullAnalyze software and were kindly provided by the GIS DRAIX.

contains peaks whose center and amplitude permit to compute the position and some characteristics of the hit target. When the footprint of the laser beam is around 1m on the Earth surface, the beam can hit multiple targets during the two-way propagation (for example the ground and the top of a tree or building). The sum of the contributions of each target hit by the laser beam then produces a complex signal with multiple peaks, each one containing information about one target.

One state of the art method to extract information from these data is to decompose them in a sum of Gaussian functions where each function represents the contribution of a target hit by the laser beam.

Therefore, we use the `scipy.optimize` module to fit a waveform to one or a sum of Gaussian functions.

Loading and visualization

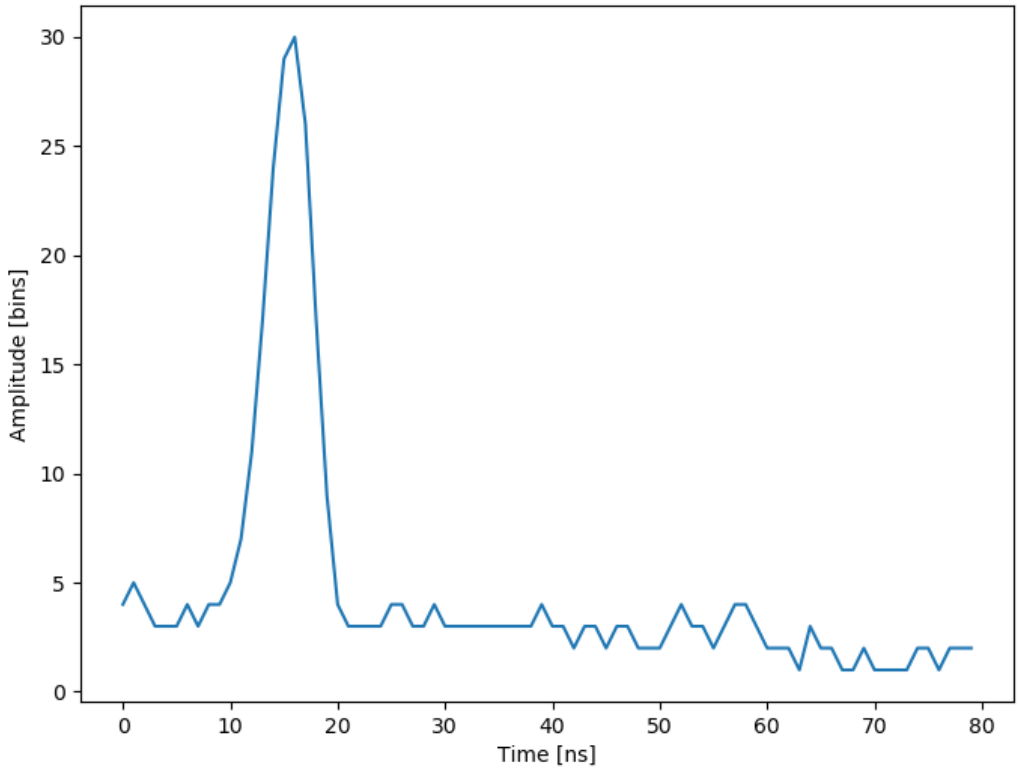
Load the first waveform using:

```
>>> import numpy as np
>>> waveform_1 = np.load('intro/summary-exercises/examples/waveform_1.npy')
```

and visualize it:

```
>>> import matplotlib.pyplot as plt
>>> t = np.arange(len(waveform_1))
>>> plt.plot(t, waveform_1)
[<matplotlib.lines.Line2D object at ...>]
>>> plt.show()
```

As shown below, this waveform is a 80-bin-length signal with a single peak with an amplitude of approximately 30 in the 15 nanosecond bin. Additionally, the base level of noise is approximately 3. These values can be used in the initial solution.



Fitting a waveform with a simple Gaussian model

The signal is very simple and can be modeled as a single Gaussian function and an offset corresponding to the background noise. To fit the signal with the function, we must:

- define the model
- propose an initial solution
- call `scipy.optimize.leastsq`

Model

A Gaussian function defined by

$$B + A \exp\left\{-\left(\frac{t-\mu}{\sigma}\right)^2\right\}$$

can be defined in python by:

```
>>> def model(t, coeffs):
...     return coeffs[0] + coeffs[1] * np.exp( - ((t-coeffs[2])/coeffs[3])**2 )
```

where

- `coeffs[0]` is B (noise)
- `coeffs[1]` is A (amplitude)
- `coeffs[2]` is μ (center)
- `coeffs[3]` is σ (width)

Initial solution

One possible initial solution that we determine by inspection is:

```
>>> x0 = np.array([3, 30, 15, 1], dtype=float)
```

Fit

`scipy.optimize.leastsq` minimizes the sum of squares of the function given as an argument. Basically, the function to minimize is the residuals (the difference between the data and the model):

```
>>> def residuals(coeffs, y, t):
...     return y - model(t, coeffs)
```

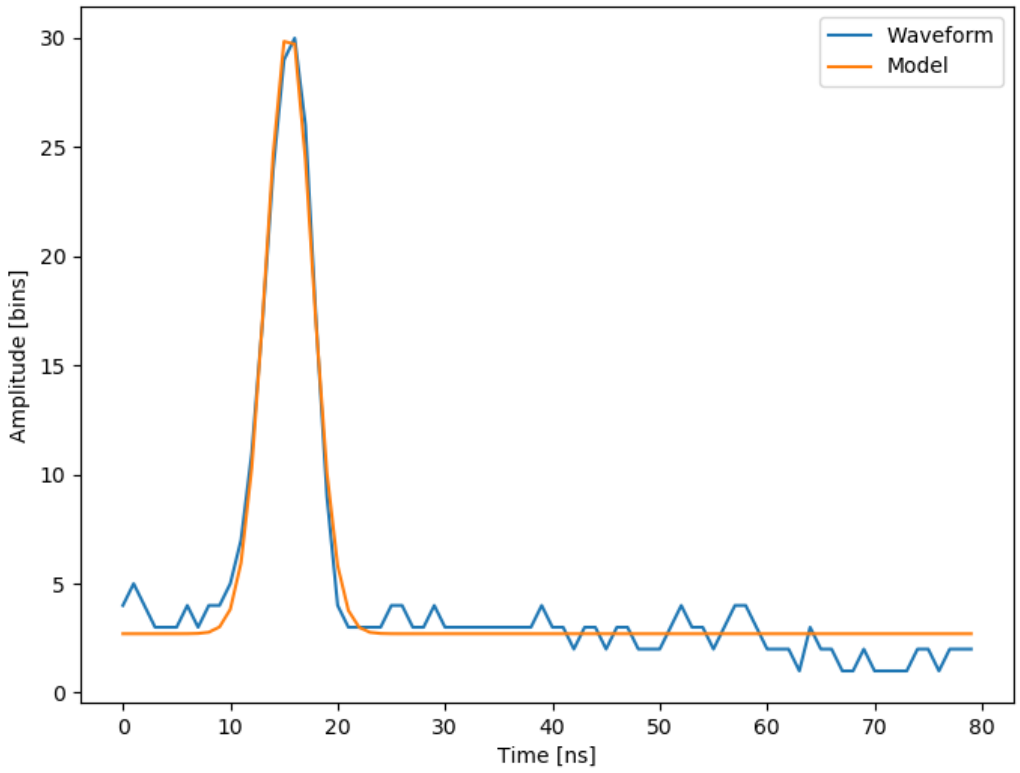
So let's get our solution by calling `scipy.optimize.leastsq()` with the following arguments:

- the function to minimize
- an initial solution
- the additional arguments to pass to the function

```
>>> from scipy.optimize import leastsq
>>> t = np.arange(len(waveform_1))
>>> x, flag = leastsq(residuals, x0, args=(waveform_1, t))
>>> print(x)
[  2.70363341  27.82020742  15.47924562   3.05636228]
```

And visualize the solution:

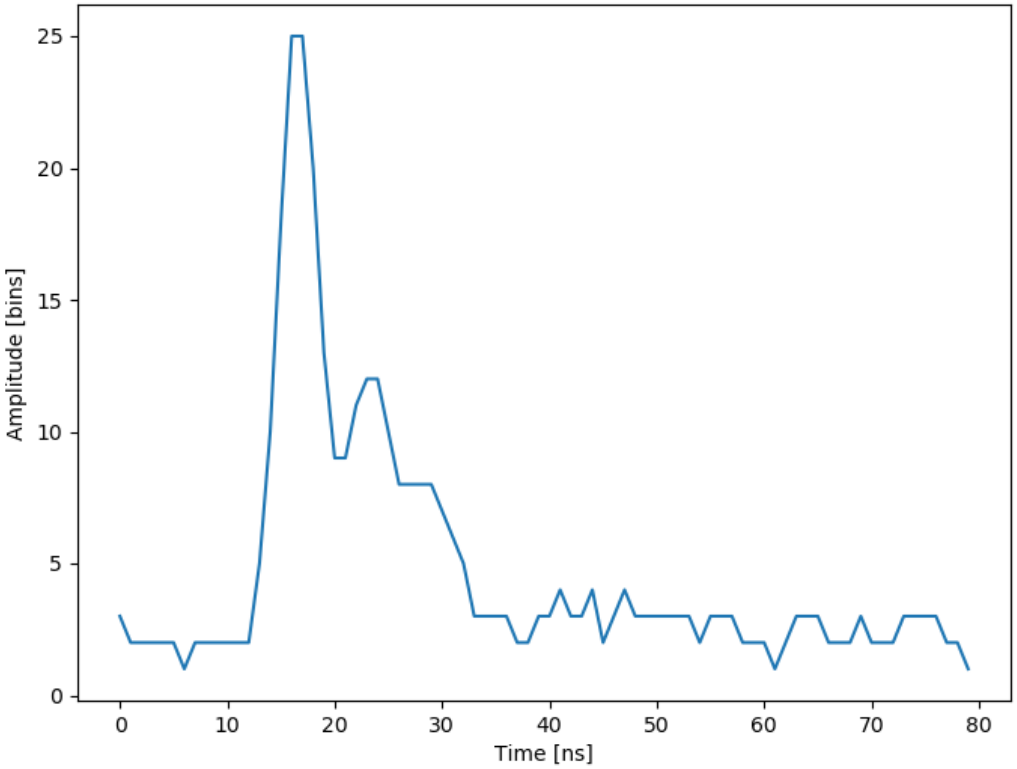
```
fig, ax = plt.subplots(figsize=(8, 6))
plt.plot(t, waveform_1, t, model(t, x))
plt.xlabel('Time [ns]')
plt.ylabel('Amplitude [bins]')
plt.legend(['Waveform', 'Model'])
plt.show()
```



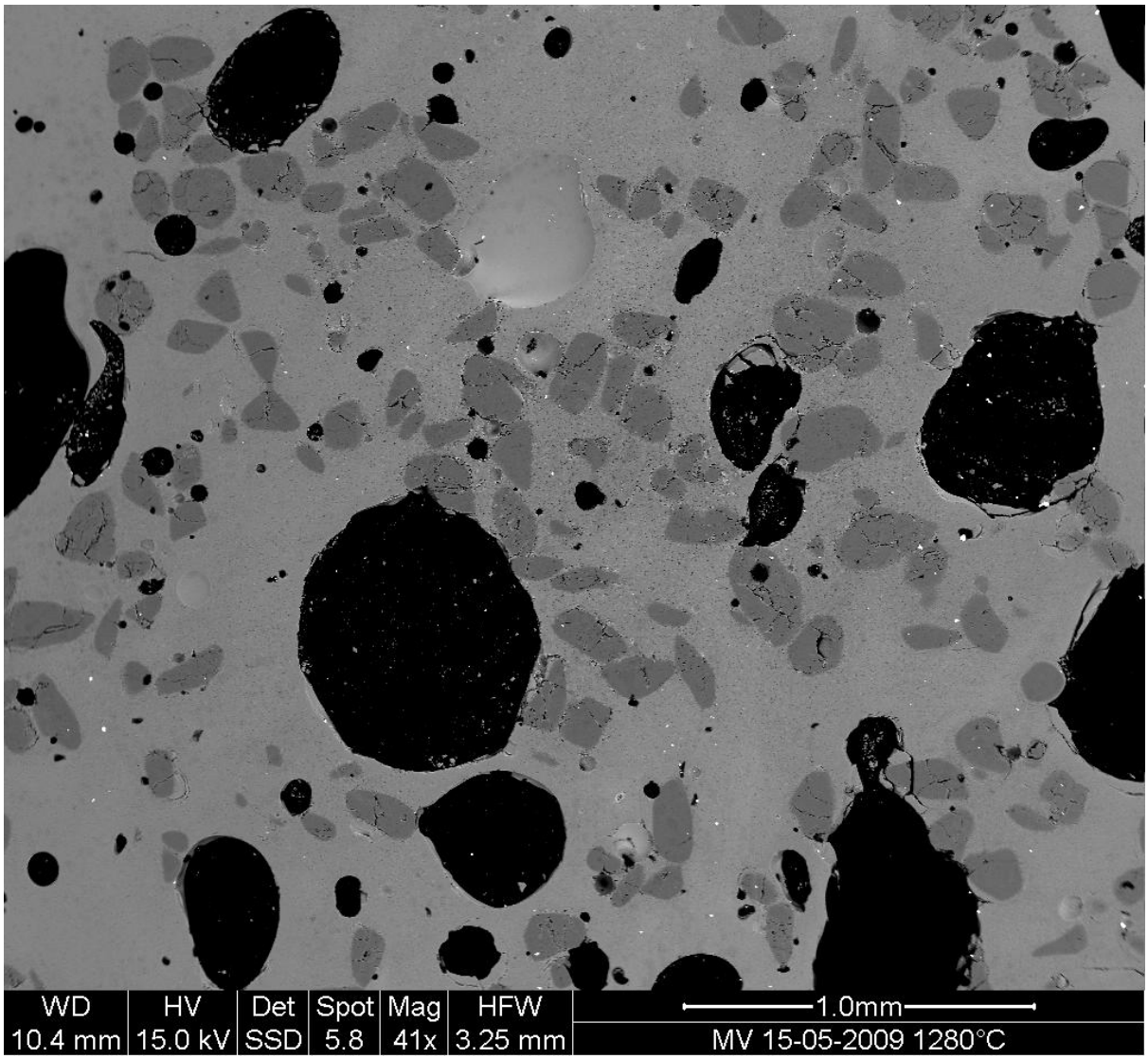
Remark: from scipy v0.8 and above, you should rather use `scipy.optimize.curve_fit()` which takes the model and the data as arguments, so you don't need to define the residuals any more.

Going further

- Try with a more complex waveform (for instance `waveform_2.npy`) that contains three significant peaks. You must adapt the model which is now a sum of Gaussian functions instead of only one Gaussian peak.
- In some cases, writing an explicit function to compute the Jacobian is faster than letting `leastsq` estimate it numerically. Create a function to compute the Jacobian of the residuals and use it as an input for `leastsq`.
- When we want to detect very small peaks in the signal, or when the initial guess is too far from a good solution, the result given by the algorithm is often not satisfying. Adding constraints to the parameters of the model enables to overcome such limitations. An example of *a priori* knowledge we can add is the sign of our variables (which are all positive).
- See the solution.
- Further exercise: compare the result of `scipy.optimize.leastsq()` and what you can get with `scipy.optimize.fmin_slsqp()` when adding boundary constraints.


Amplitude [bins]
Time [ns]
0
5
10
15
20
25
0
10
20
30
40
50
60
70
80

6.11.3 Image processing application: counting bubbles and unmolten grains



Statement of the problem

1. Open the image file MV_HFV_012.jpg and display it. Browse through the keyword arguments in the docstring of `imshow` to display the image with the "right" orientation (origin in the bottom left corner, and not the upper left corner as for standard arrays).

   This Scanning Element Microscopy image shows a glass sample (light gray matrix) with some bubbles (on black) and unmolten sand grains (dark gray). We wish to determine the fraction of the sample covered by these three phases, and to estimate the typical size of sand grains and bubbles, their sizes, etc.

2. Crop the image to remove the lower panel with measure information.

3. Slightly filter the image with a median filter in order to refine its histogram. Check how the histogram changes.

4. Using the histogram of the filtered image, determine thresholds that allow to define masks for sand pixels, glass pixels and bubble pixels. Other option (homework): write a function that determines automatically the thresholds from the minima of the histogram.

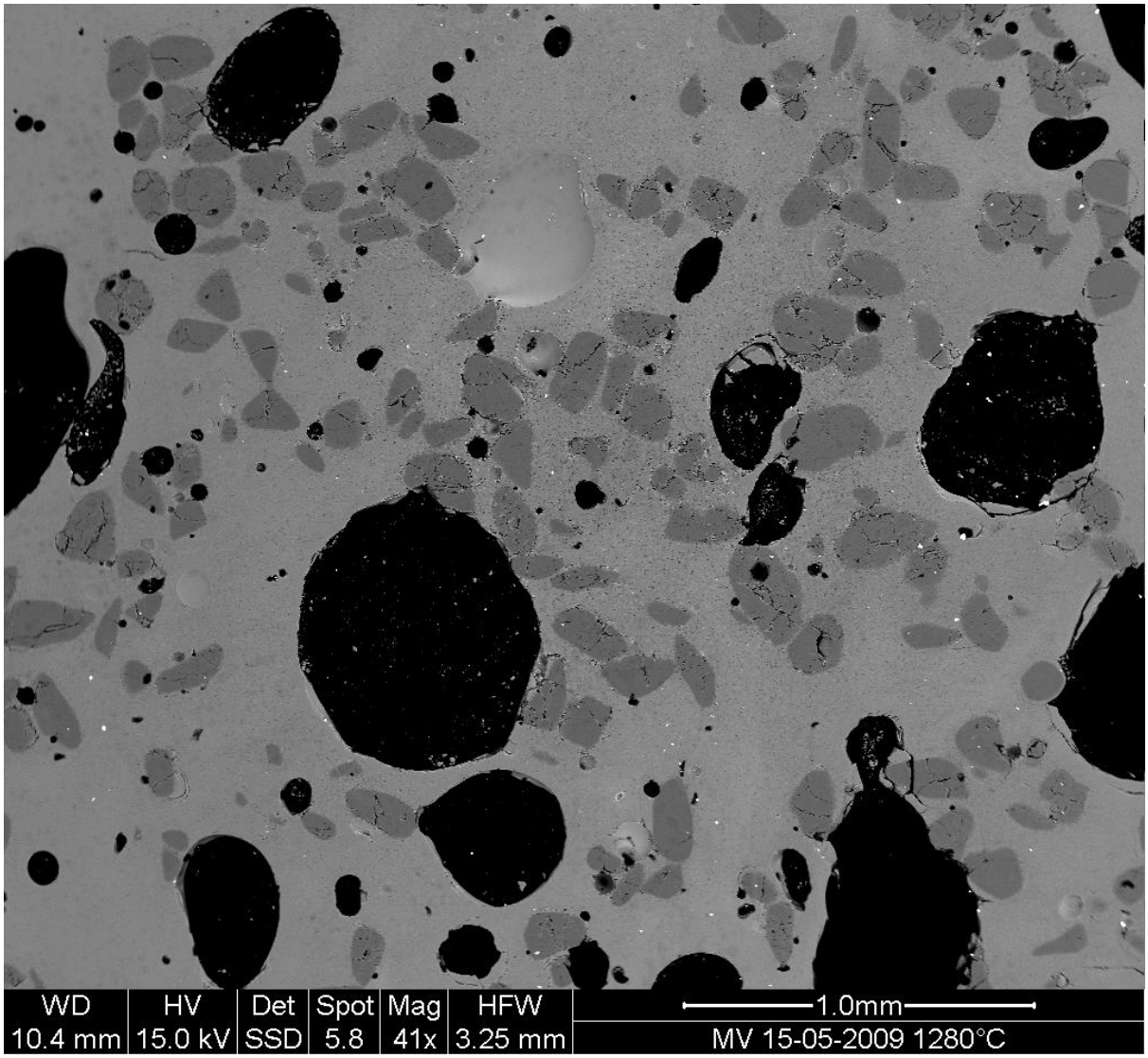
5. Display an image in which the three phases are colored with three different colors.

6. Use mathematical morphology to clean the different phases.

7. Attribute labels to all bubbles and sand grains, and remove from the sand mask grains that are smaller than 10 pixels. To do so, use `ndimage.sum` or `np.bincount` to compute the grain sizes.

8. Compute the mean size of bubbles.

6.11.4 Example of solution for the image processing exercise: unmolten grains in glass



1. Open the image file MV_HFV_012.jpg and display it. Browse through the keyword arguments in the docstring of `imshow` to display the image with the "right" orientation (origin in the bottom left corner, and not the upper left corner as for standard arrays).

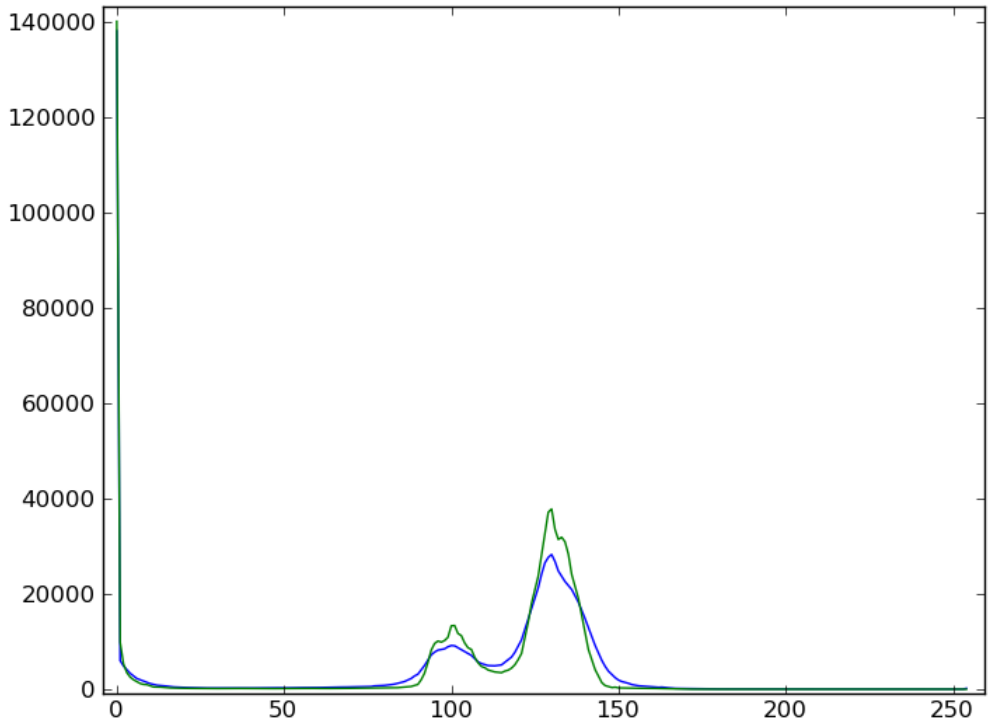
```
>>> dat = plt.imread('data/MV_HFV_012.jpg')
```

2. Crop the image to remove the lower panel with measure information.

```
>>> dat = dat[:-60]
```

3. Slightly filter the image with a median filter in order to refine its histogram. Check how the histogram changes.

```
>>> filtdat = ndimage.median_filter(dat, size=(7,7))
>>> hi_dat = np.histogram(dat, bins=np.arange(256))
>>> hi_filtdat = np.histogram(filtdat, bins=np.arange(256))
```

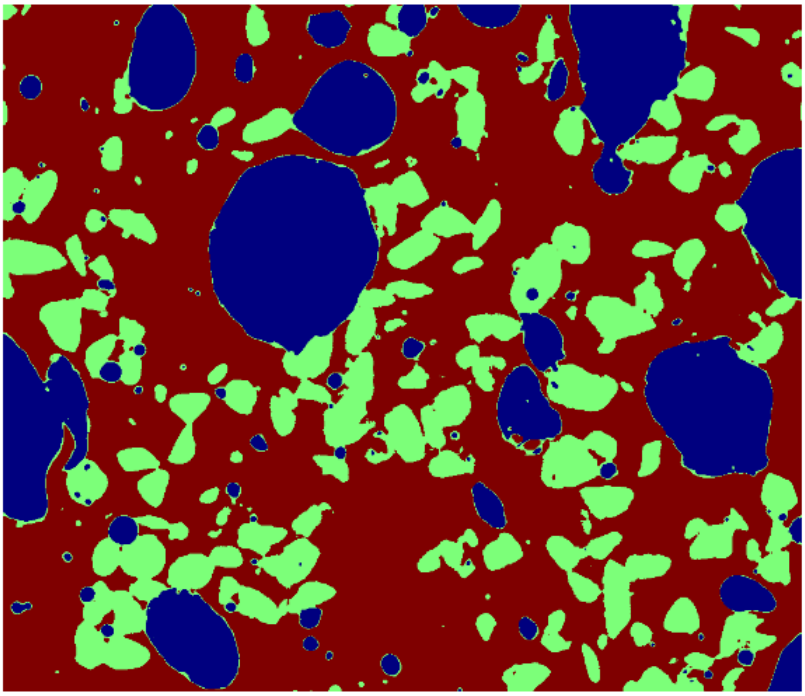


4. Using the histogram of the filtered image, determine thresholds that allow to define masks for sand pixels, glass pixels and bubble pixels. Other option (homework): write a function that determines automatically the thresholds from the minima of the histogram.

```
>>> void = filtdat <= 50
>>> sand = np.logical_and(filtdat > 50, filtdat <= 114)
>>> glass = filtdat > 114
```

5. Display an image in which the three phases are colored with three different colors.

```
>>> phases = void.astype(np.int) + 2*glass.astype(np.int) + 3*sand.astype(np.int)
```

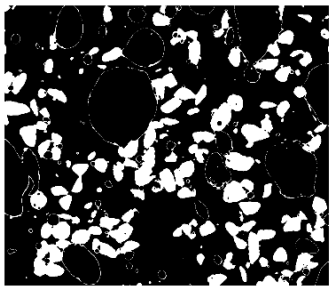
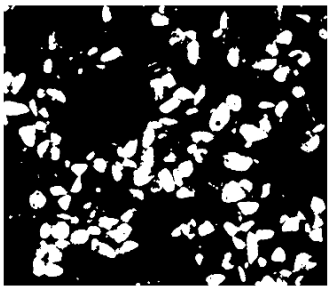
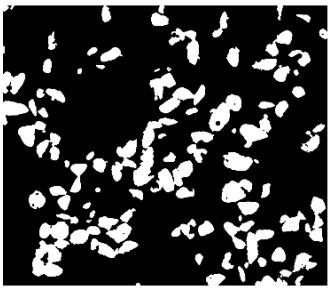


6. Use mathematical morphology to clean the different phases.

```
>>> sand_op = ndimage.binary_opening(sand, iterations=2)
```

7. Attribute labels to all bubbles and sand grains, and remove from the sand mask grains that are smaller than 10 pixels. To do so, use `ndimage.sum` or `np.bincount` to compute the grain sizes.

```
>>> sand_labels, sand_nb = ndimage.label(sand_op)
>>> sand_areas = np.array(ndimage.sum(sand_op, sand_labels, np.arange(sand_labels.
↪max()+1)))
>>> mask = sand_areas > 100
>>> remove_small_sand = mask[sand_labels.ravel()].reshape(sand_labels.shape)
```

8. Compute the mean size of bubbles.

```
>>> bubbles_labels, bubbles_nb = ndimage.label(void)
>>> bubbles_areas = np.bincount(bubbles_labels.ravel())[1:]
>>> mean_bubble_size = bubbles_areas.mean()
>>> median_bubble_size = np.median(bubbles_areas)
>>> mean_bubble_size, median_bubble_size
```

(continues on next page)

(continued from previous page)

```
(1699.875, 65.0)
```

6.12 Full code examples for the scipy chapter

Note: Click *here* to download the full example code

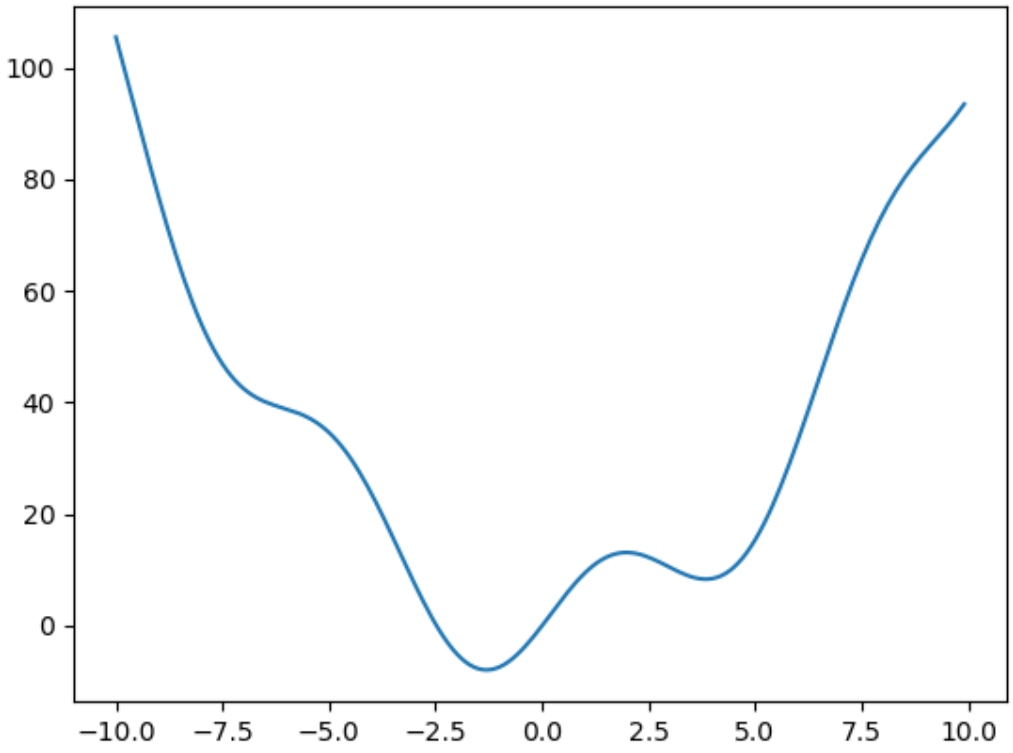
6.12.1 Finding the minimum of a smooth function

Demos various methods to find the minimum of a function.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**2 + 10*np.sin(x)


x = np.arange(-10, 10, 0.1)
plt.plot(x, f(x))
```



Now find the minimum with a few methods

```
from scipy import optimize

# The default (Nelder Mead)
print(optimize.minimize(f, x0=0))
```

Out:

```
fun: -7.945823375615215
 hess_inv: array([[0.08589237]])
      jac: array([-1.1920929e-06])
  message: 'Optimization terminated successfully.'
     nfev: 18
      nit: 5
     njev: 6
   status: 0
  success: True
        x: array([-1.30644012])
```

```
print(optimize.minimize(f, x0=0, method="L-BFGS-B"))
```

Out:

```
fun: array([-7.94582338])
 hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
      jac: array([-1.42108547e-06])
  message: b'CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL'
     nfev: 12
      nit: 5
   status: 0
  success: True
        x: array([-1.30644013])
```

```
plt.show()
```

Total running time of the script: (0 minutes 0.016 seconds)

Note: Click *here* to download the full example code

6.12.2 Detrending a signal

`scipy.signal.detrend()` removes a linear trend.

Generate a random signal with a trend

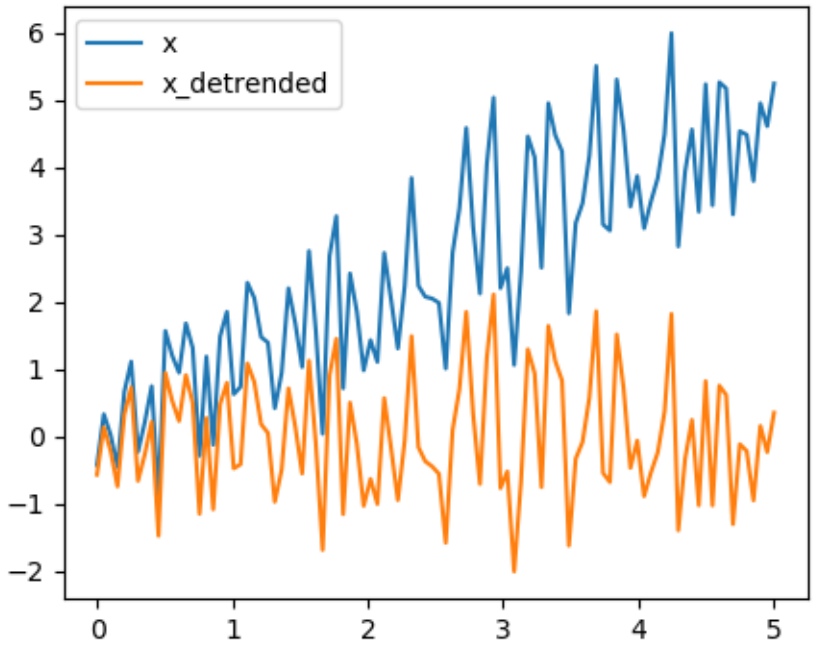
```
import numpy as np
t = np.linspace(0, 5, 100)
x = t + np.random.normal(size=100)
```

Detrend

```
from scipy import signal
x_detrended = signal.detrend(x)
```

Plot

```
from matplotlib import pyplot as plt
plt.figure(figsize=(5, 4))
plt.plot(t, x, label="x")
plt.plot(t, x_detrended, label="x_detrended")
plt.legend(loc='best')
plt.show()
```



Total running time of the script: (0 minutes 0.168 seconds)

Note: Click *here* to download the full example code

6.12.3 Resample a signal with scipy.signal.resample

`scipy.signal.resample()` uses FFT to resample a 1D signal.

Generate a signal with 100 data point

```
import numpy as np
t = np.linspace(0, 5, 100)
x = np.sin(t)
```

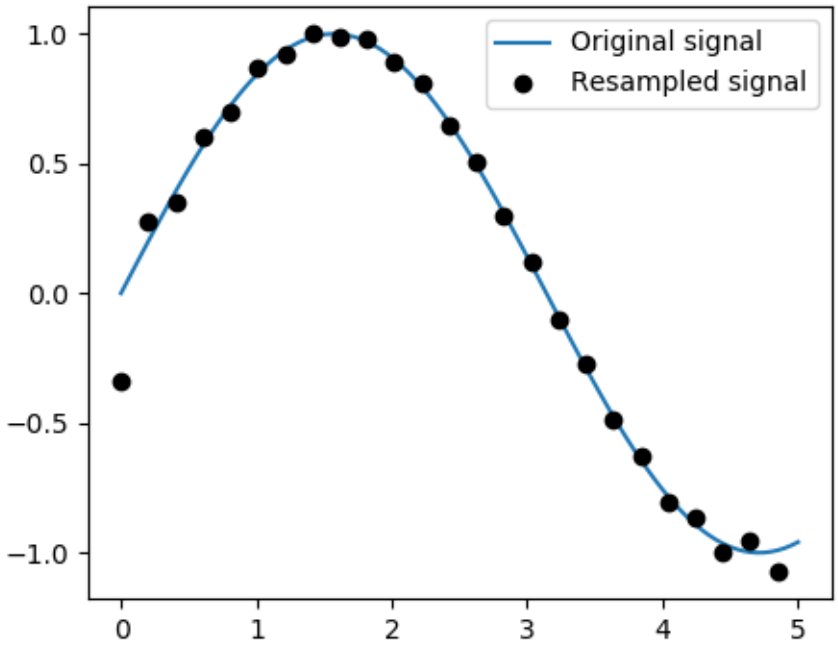
Downsample it by a factor of 4

```
from scipy import signal
x_resampled = signal.resample(x, 25)
```

Plot

```
from matplotlib import pyplot as plt
plt.figure(figsize=(5, 4))
plt.plot(t, x, label='Original signal')
plt.plot(t[::4], x_resampled, 'ko', label='Resampled signal')

plt.legend(loc='best')
plt.show()
```

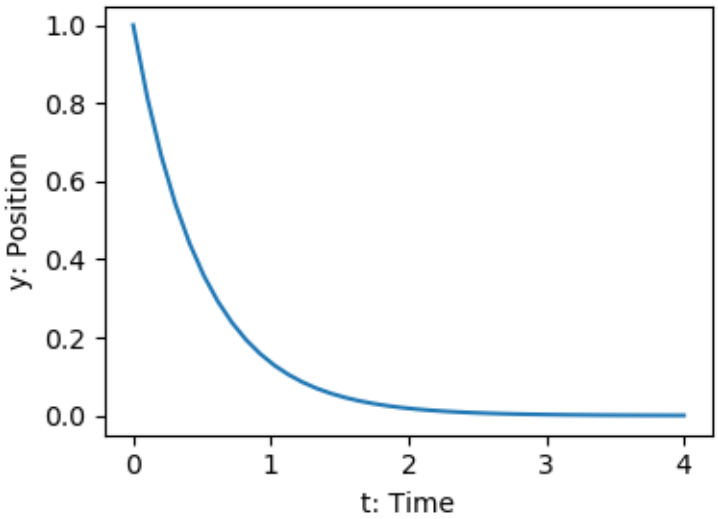


Total running time of the script: (0 minutes 0.019 seconds)

Note: Click *here* to download the full example code

6.12.4 Integrating a simple ODE

Solve the ODE dy/dt = -2y between t = 0..4, with the initial condition y(t=0) = 1.




```
import numpy as np
from scipy.integrate import odeint
from matplotlib import pyplot as plt

def calc_derivative(ypos, time):
```

(continues on next page)

(continued from previous page)

```
    return -2*ypos

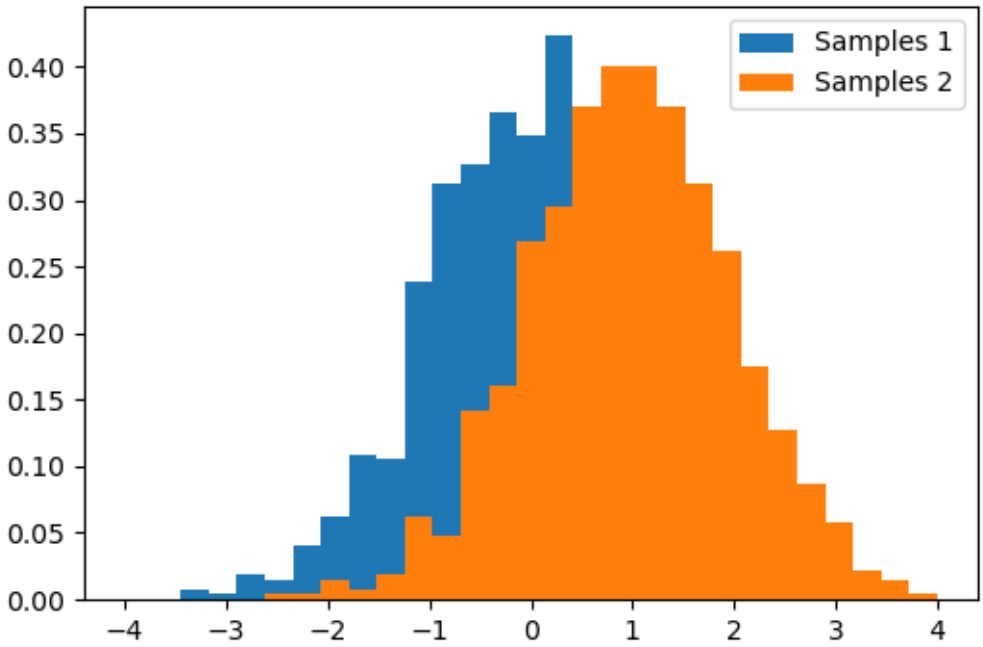
time_vec = np.linspace(0, 4, 40)
yvec = odeint(calc_derivative, 1, time_vec)

plt.figure(figsize=(4, 3))
plt.plot(time_vec, yvec)
plt.xlabel('t: Time')
plt.ylabel('y: Position')
plt.tight_layout()
```

Total running time of the script: (0 minutes 0.034 seconds)

Note: Click *here* to download the full example code

6.12.5 Comparing 2 sets of samples from Gaussians




```
import numpy as np
from matplotlib import pyplot as plt

# Generates 2 sets of observations
samples1 = np.random.normal(0, size=1000)
samples2 = np.random.normal(1, size=1000)

# Compute a histogram of the sample
bins = np.linspace(-4, 4, 30)
histogram1, bins = np.histogram(samples1, bins=bins, density=True)
histogram2, bins = np.histogram(samples2, bins=bins, density=True)

plt.figure(figsize=(6, 4))
plt.hist(samples1, bins=bins, density=True, label="Samples 1")
```

(continues on next page)

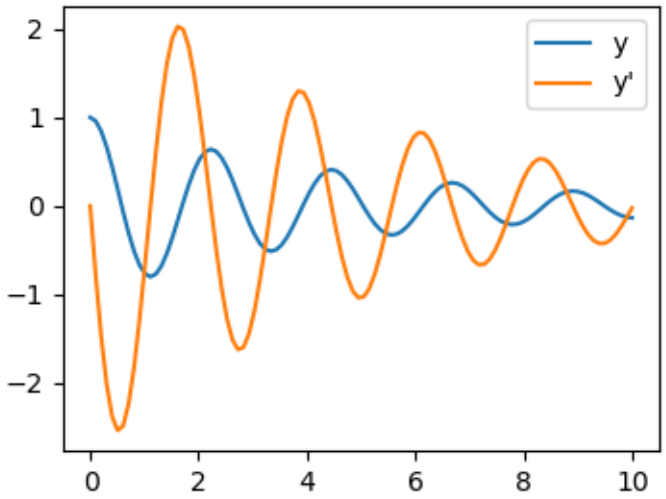
(continued from previous page)

```
plt.hist(samples2, bins=bins, density=True, label="Samples 2")
plt.legend(loc='best')
plt.show()
```

Total running time of the script: (0 minutes 0.042 seconds)

Note: Click *here* to download the full example code

6.12.6 Integrate the Damped spring-mass oscillator




```
import numpy as np
from scipy.integrate import odeint
from matplotlib import pyplot as plt

mass = 0.5  # kg
kspring = 4  # N/m
cviscous = 0.4  # N s/m


eps = cviscous / (2 * mass * np.sqrt(kspring/mass))
omega = np.sqrt(kspring / mass)


def calc_deri(yvec, time, eps, omega):
    return (yvec[1], -eps * omega * yvec[1] - omega **2 * yvec[0])

time_vec = np.linspace(0, 10, 100)
yinit = (1, 0)
yarr = odeint(calc_deri, yinit, time_vec, args=(eps, omega))

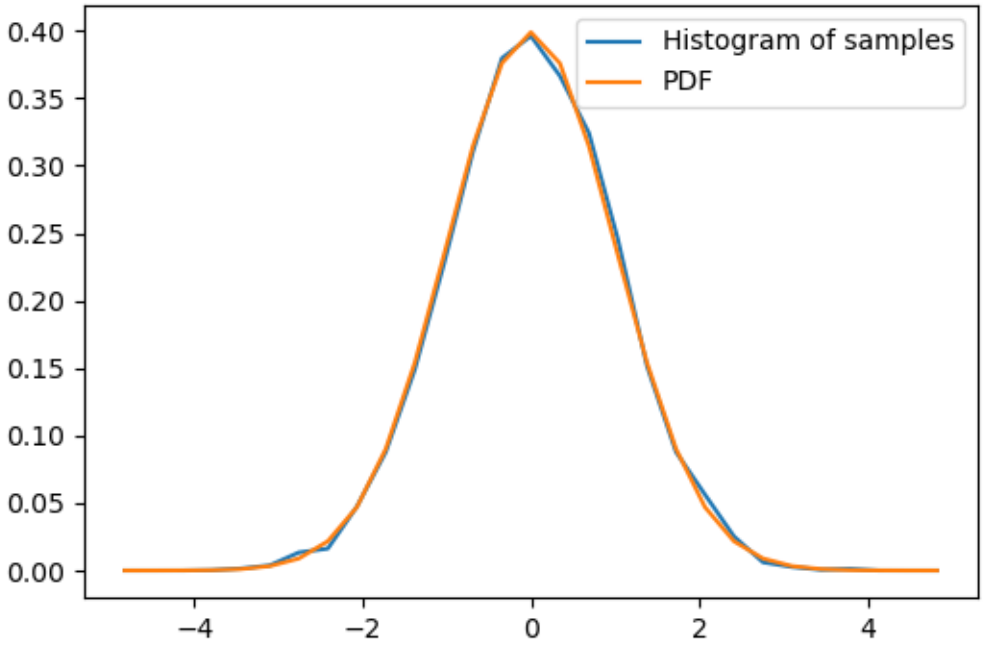
plt.figure(figsize=(4, 3))
plt.plot(time_vec, yarr[:, 0], label='y')
plt.plot(time_vec, yarr[:, 1], label="y'")
plt.legend(loc='best')
plt.show()
```

Total running time of the script: (0 minutes 0.015 seconds)

Note: Click *here* to download the full example code

6.12.7 Normal distribution: histogram and PDF

Explore the normal distribution: a histogram built from samples and the PDF (probability density function).




```
import numpy as np

# Sample from a normal distribution using numpy's random number generator
samples = np.random.normal(size=10000)

# Compute a histogram of the sample
bins = np.linspace(-5, 5, 30)
histogram, bins = np.histogram(samples, bins=bins, density=True)

bin_centers = 0.5*(bins[1:] + bins[:-1])

# Compute the PDF on the bin centers from scipy distribution object
from scipy import stats
pdf = stats.norm.pdf(bin_centers)

from matplotlib import pyplot as plt
plt.figure(figsize=(6, 4))
plt.plot(bin_centers, histogram, label="Histogram of samples")
plt.plot(bin_centers, pdf, label="PDF")
plt.legend()
plt.show()
```

Total running time of the script: (0 minutes 0.014 seconds)

Note: Click *here* to download the full example code

6.12.8 Curve fitting

Demos a simple curve fitting

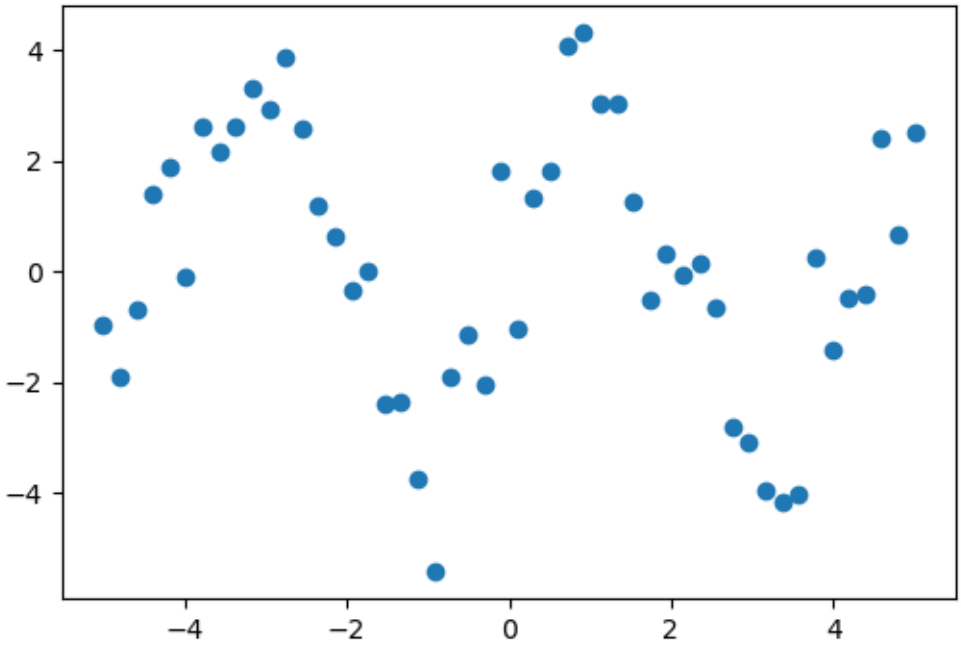
First generate some data

```
import numpy as np

# Seed the random number generator for reproducibility
np.random.seed(0)

x_data = np.linspace(-5, 5, num=50)
y_data = 2.9 * np.sin(1.5 * x_data) + np.random.normal(size=50)

# And plot it
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 4))
plt.scatter(x_data, y_data)
```



Now fit a simple sine function to the data

```
from scipy import optimize

def test_func(x, a, b):
    return a * np.sin(b * x)

params, params_covariance = optimize.curve_fit(test_func, x_data, y_data,
                                               p0=[2, 2])

print(params)
```

Out:

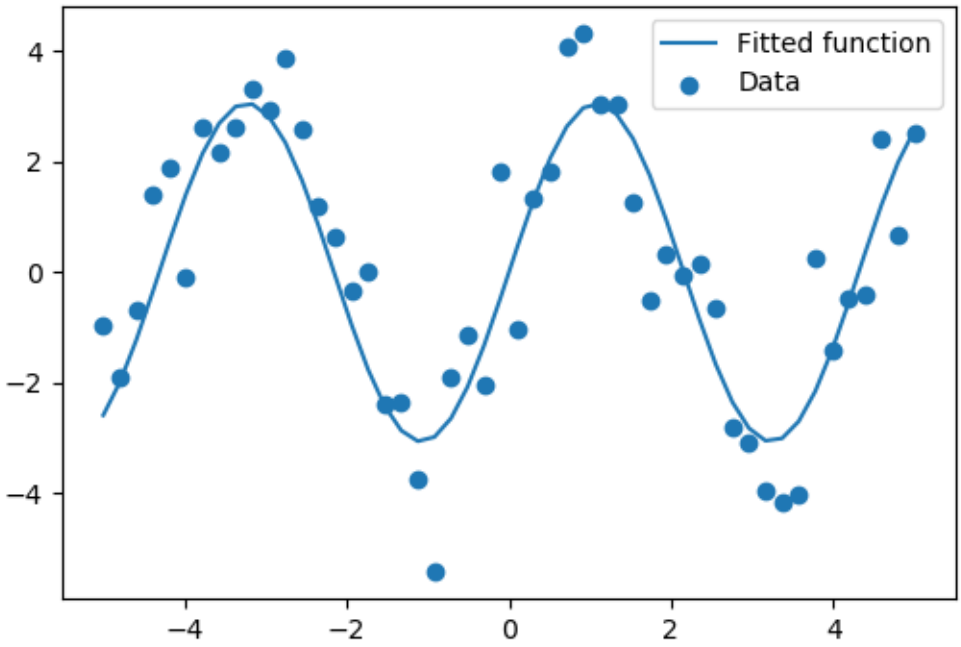
```
[3.05931973 1.45754553]
```

And plot the resulting curve on the data

```
plt.figure(figsize=(6, 4))
plt.scatter(x_data, y_data, label='Data')
plt.plot(x_data, test_func(x_data, params[0], params[1]),
         label='Fitted function')

plt.legend(loc='best')

plt.show()
```



Total running time of the script: (0 minutes 0.026 seconds)

Note: Click *here* to download the full example code

6.12.9 Spectrogram, power spectral density

Demo spectrogram and power spectral density on a frequency chirp.

```
import numpy as np
from matplotlib import pyplot as plt
```

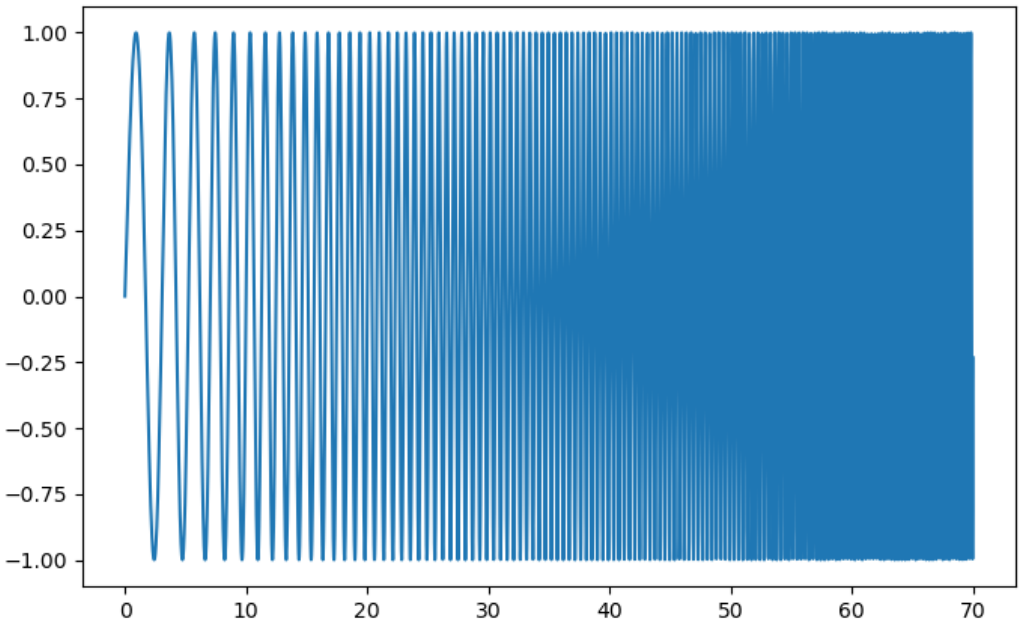
Generate a chirp signal

```
# Seed the random number generator
np.random.seed(0)

time_step = .01
time_vec = np.arange(0, 70, time_step)

# A signal with a small frequency chirp
sig = np.sin(0.5 * np.pi * time_vec * (1 + .1 * time_vec))

plt.figure(figsize=(8, 5))
plt.plot(time_vec, sig)
```

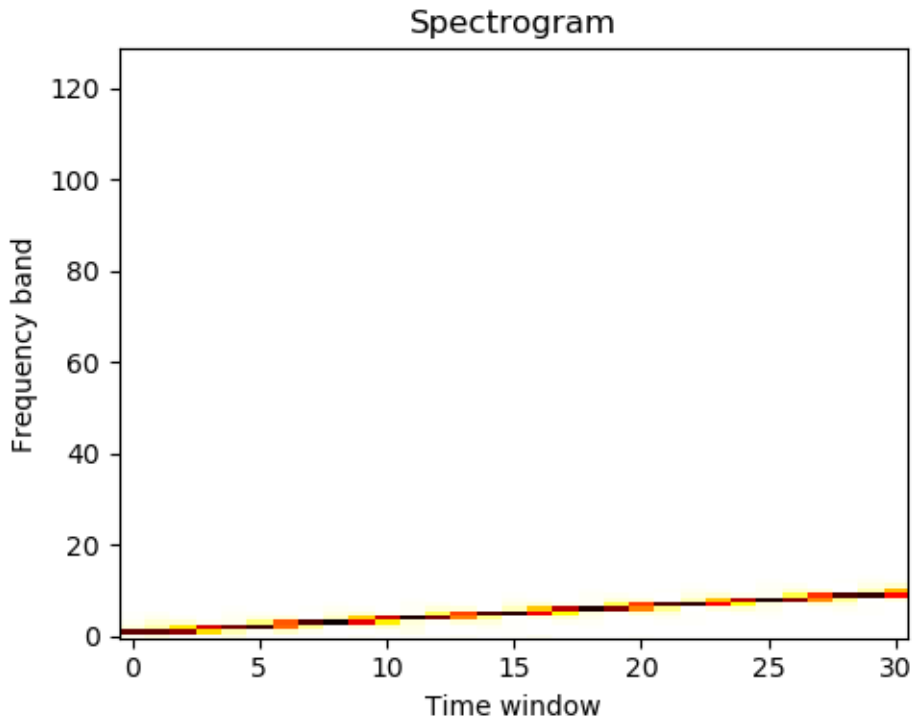


Compute and plot the spectrogram

The spectrum of the signal on consecutive time windows

```
from scipy import signal
freqs, times, spectrogram = signal.spectrogram(sig)

plt.figure(figsize=(5, 4))
plt.imshow(spectrogram, aspect='auto', cmap='hot_r', origin='lower')
plt.title('Spectrogram')
plt.ylabel('Frequency band')
plt.xlabel('Time window')
plt.tight_layout()
```

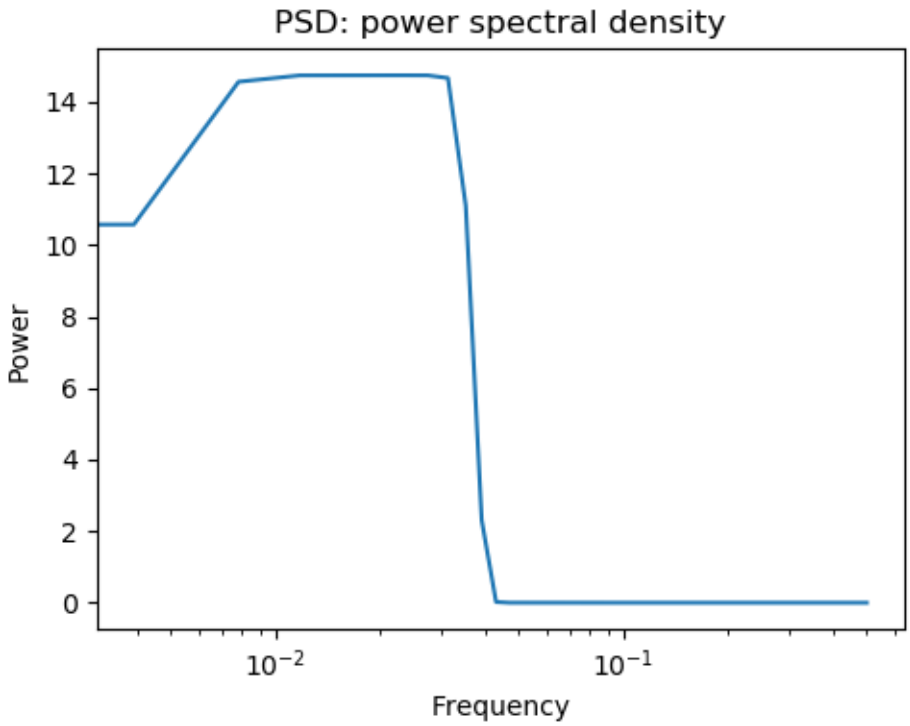


Compute and plot the power spectral density (PSD)

The power of the signal per frequency band

```
freqs, psd = signal.welch(sig)

plt.figure(figsize=(5, 4))
plt.semilogx(freqs, psd)
plt.title('PSD: power spectral density')
plt.xlabel('Frequency')
plt.ylabel('Power')
plt.tight_layout()
```

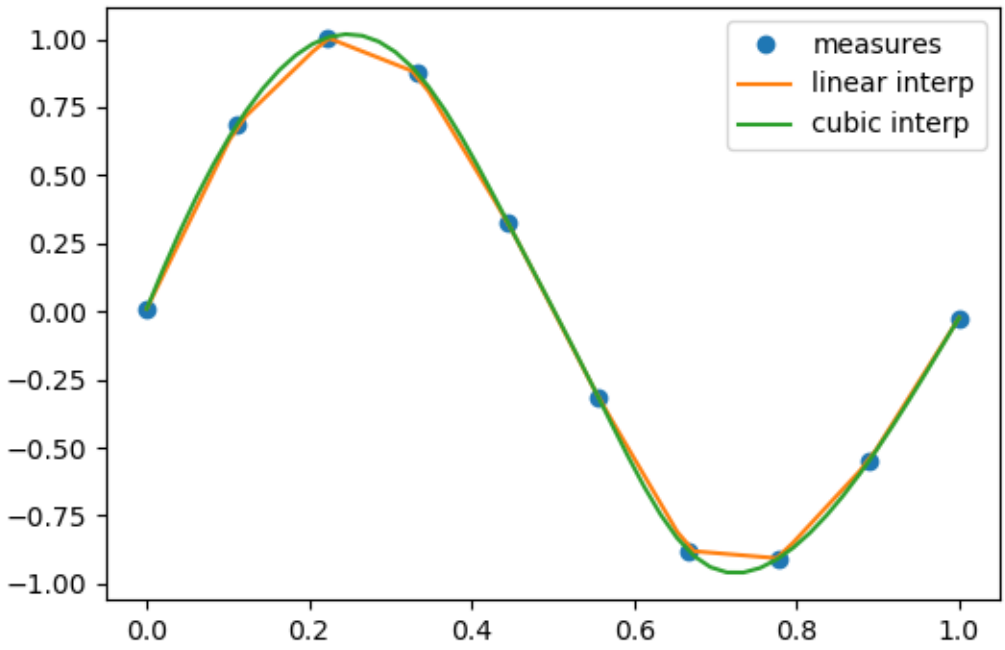



```
plt.show()
```

Total running time of the script: (0 minutes 0.200 seconds)

Note: Click *here* to download the full example code

6.12.10 A demo of 1D interpolation




```
# Generate data
import numpy as np
np.random.seed(0)
measured_time = np.linspace(0, 1, 10)
noise = 1e-1 * (np.random.random(10)*2 - 1)
measures = np.sin(2 * np.pi * measured_time) + noise

# Interpolate it to new time points
from scipy.interpolate import interp1d
linear_interp = interp1d(measured_time, measures)
interpolation_time = np.linspace(0, 1, 50)
linear_results = linear_interp(interpolation_time)
cubic_interp = interp1d(measured_time, measures, kind='cubic')
cubic_results = cubic_interp(interpolation_time)

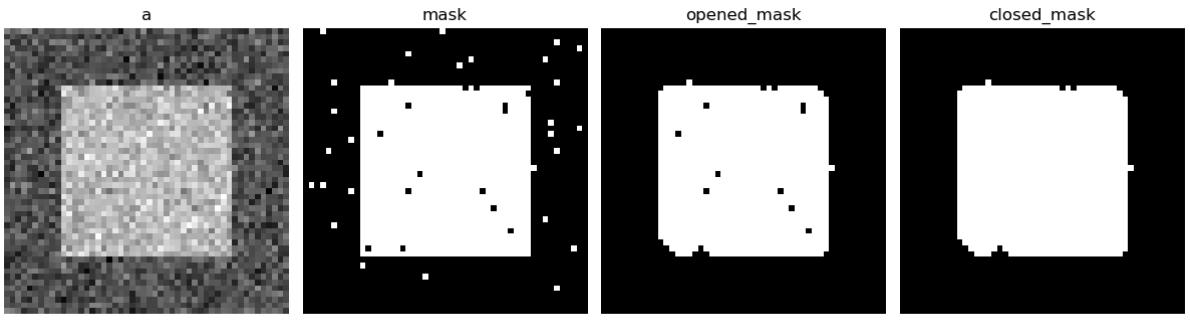
# Plot the data and the interpolation
from matplotlib import pyplot as plt
plt.figure(figsize=(6, 4))
plt.plot(measured_time, measures, 'o', ms=6, label='measures')
plt.plot(interpolation_time, linear_results, label='linear interp')
plt.plot(interpolation_time, cubic_results, label='cubic interp')
plt.legend()
plt.show()
```

Total running time of the script: (0 minutes 0.018 seconds)

Note: Click *here* to download the full example code

6.12.11 Demo mathematical morphology

A basic demo of binary opening and closing.




```
# Generate some binary data
import numpy as np
np.random.seed(0)
a = np.zeros((50, 50))
a[10:-10, 10:-10] = 1
a += 0.25 * np.random.standard_normal(a.shape)
mask = a>=0.5

# Apply mathematical morphology
from scipy import ndimage
opened_mask = ndimage.binary_opening(mask)
closed_mask = ndimage.binary_closing(opened_mask)

# Plot
from matplotlib import pyplot as plt

plt.figure(figsize=(12, 3.5))
plt.subplot(141)
plt.imshow(a, cmap=plt.cm.gray)
plt.axis('off')
plt.title('a')

plt.subplot(142)
plt.imshow(mask, cmap=plt.cm.gray)
plt.axis('off')
plt.title('mask')

plt.subplot(143)
plt.imshow(opened_mask, cmap=plt.cm.gray)
plt.axis('off')
plt.title('opened_mask')

plt.subplot(144)
plt.imshow(closed_mask, cmap=plt.cm.gray)
plt.title('closed_mask')
plt.axis('off')

plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.99)

plt.show()
```

Total running time of the script: (0 minutes 0.041 seconds)

Note: Click *here* to download the full example code

6.12.12 Plot geometrical transformations on images

Demo geometrical transformations of images.



```
# Load some data
from scipy import misc
face = misc.face(gray=True)

# Apply a variety of transformations
from scipy import ndimage
from matplotlib import pyplot as plt
shifted_face = ndimage.shift(face, (50, 50))
shifted_face2 = ndimage.shift(face, (50, 50), mode='nearest')
rotated_face = ndimage.rotate(face, 30)
cropped_face = face[50:-50, 50:-50]
zoomed_face = ndimage.zoom(face, 2)
zoomed_face.shape

plt.figure(figsize=(15, 3))
plt.subplot(151)
plt.imshow(shifted_face, cmap=plt.cm.gray)
plt.axis('off')

plt.subplot(152)
plt.imshow(shifted_face2, cmap=plt.cm.gray)
plt.axis('off')

plt.subplot(153)
plt.imshow(rotated_face, cmap=plt.cm.gray)
plt.axis('off')

plt.subplot(154)
plt.imshow(cropped_face, cmap=plt.cm.gray)
plt.axis('off')

plt.subplot(155)
plt.imshow(zoomed_face, cmap=plt.cm.gray)
plt.axis('off')

plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.99)

plt.show()
```

Total running time of the script: (0 minutes 0.773 seconds)

Note: Click *here* to download the full example code

6.12.13 Demo connected components

Extracting and labeling connected components in a 2D array

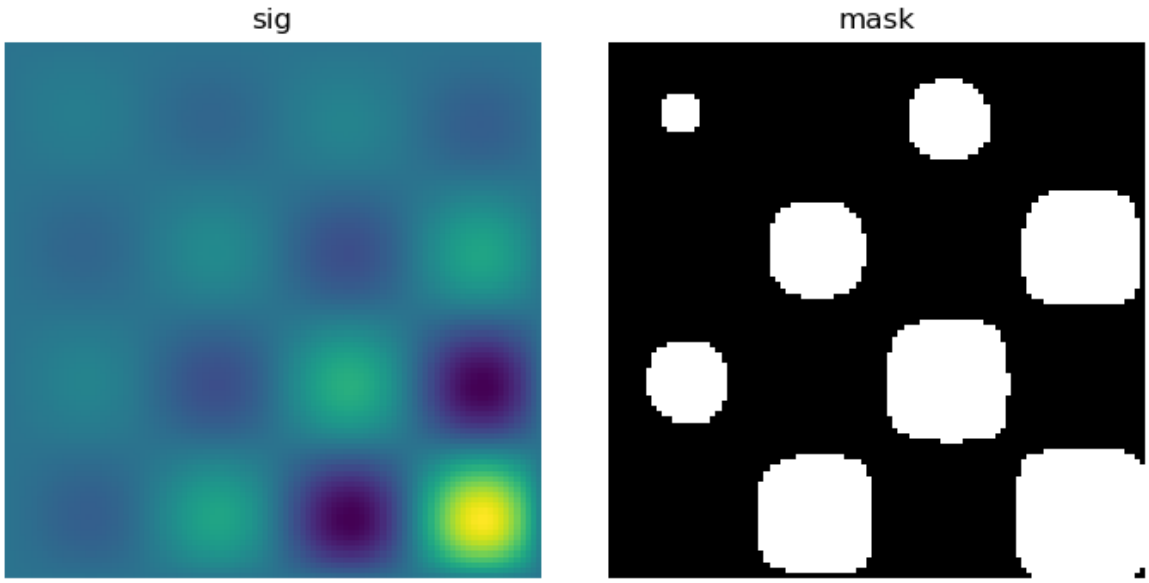
```
import numpy as np
from matplotlib import pyplot as plt
```

Generate some binary data

```
np.random.seed(0)
x, y = np.indices((100, 100))
sig = np.sin(2*np.pi*x/50.) * np.sin(2*np.pi*y/50.) * (1+x*y/50.**2)**2
mask = sig > 1

plt.figure(figsize=(7, 3.5))
plt.subplot(1, 2, 1)
plt.imshow(sig)
plt.axis('off')
plt.title('sig')

plt.subplot(1, 2, 2)
plt.imshow(mask, cmap=plt.cm.gray)
plt.axis('off')
plt.title('mask')
plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.9)
```

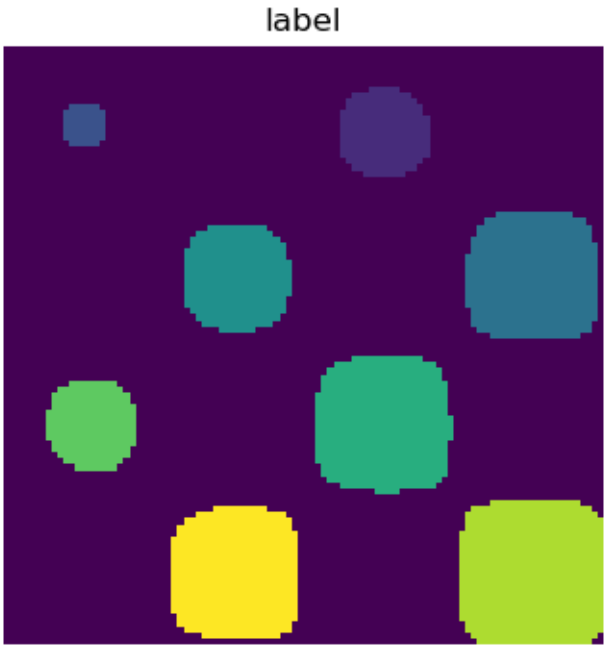


Label connected components

```
from scipy import ndimage
labels, nb = ndimage.label(mask)

plt.figure(figsize=(3.5, 3.5))
plt.imshow(labels)
plt.title('label')
plt.axis('off')

plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.9)
```

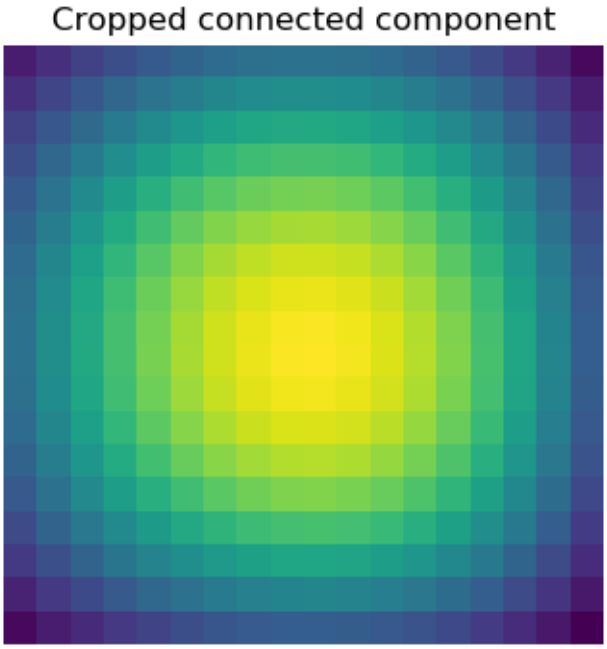


Extract the 4th connected component, and crop the array around it

```
sl = ndimage.find_objects(labels==4)
plt.figure(figsize=(3.5, 3.5))
plt.imshow(sig[sl[0]])
plt.title('Cropped connected component')
plt.axis('off')

plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.9)

plt.show()
```



Total running time of the script: (0 minutes 0.047 seconds)

Note: Click *here* to download the full example code

6.12.14 Minima and roots of a function

Demos finding minima and roots of a function.

Define the function

```
import numpy as np

x = np.arange(-10, 10, 0.1)
def f(x):
    return x**2 + 10*np.sin(x)
```

Find minima

```
from scipy import optimize

# Global optimization
grid = (-10, 10, 0.1)
xmin_global = optimize.brute(f, (grid, ))
print("Global minima found %s" % xmin_global)

# Constrain optimization
xmin_local = optimize.fminbound(f, 0, 10)
print("Local minimum found %s" % xmin_local)
```

Out:

```
Global minima found [-1.30641113]
Local minimum found 3.8374671194983834
```

Root finding

```
root = optimize.root(f, 1)  # our initial guess is 1
print("First root found %s" % root.x)
root2 = optimize.root(f, -2.5)
print("Second root found %s" % root2.x)
```

Out:

```
First root found [0.]
Second root found [-2.47948183]
```

Plot function, minima, and roots

```
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)

# Plot the function
ax.plot(x, f(x), 'b-', label="f(x)")

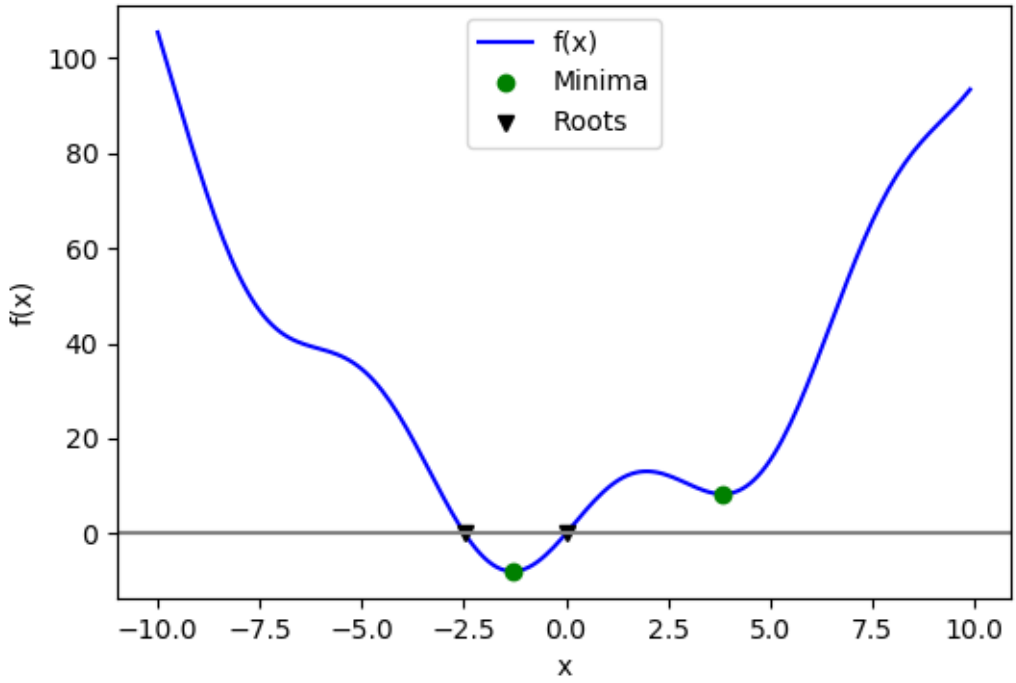
# Plot the minima
xmins = np.array([xmin_global[0], xmin_local])
ax.plot(xmins, f(xmins), 'go', label="Minima")
```

(continues on next page)

(continued from previous page)

```
# Plot the roots
roots = np.array([root.x, root2.x])
ax.plot(roots, f(roots), 'kv', label="Roots")

# Decorate the figure
ax.legend(loc='best')
ax.set_xlabel('x')
ax.set_ylabel('f(x)')
ax.axhline(0, color='gray')
plt.show()
```



Total running time of the script: (0 minutes 0.084 seconds)

Note: Click *here* to download the full example code

6.12.15 Plot filtering on images

Demo filtering for denoising of images.



```
# Load some data
from scipy import misc
face = misc.face(gray=True)
face = face[:512, -512:]  # crop out square on right

# Apply a variety of filters
from scipy import ndimage
from scipy import signal
from matplotlib import pyplot as plt

import numpy as np
noisy_face = np.copy(face).astype(np.float)
noisy_face += face.std() * 0.5 * np.random.standard_normal(face.shape)
blurred_face = ndimage.gaussian_filter(noisy_face, sigma=3)
median_face = ndimage.median_filter(noisy_face, size=5)
wiener_face = signal.wiener(noisy_face, (5, 5))

plt.figure(figsize=(12, 3.5))
plt.subplot(141)
plt.imshow(noisy_face, cmap=plt.cm.gray)
plt.axis('off')
plt.title('noisy')

plt.subplot(142)
plt.imshow(blurred_face, cmap=plt.cm.gray)
plt.axis('off')
plt.title('Gaussian filter')

plt.subplot(143)
plt.imshow(median_face, cmap=plt.cm.gray)
plt.axis('off')
plt.title('median filter')

plt.subplot(144)
plt.imshow(wiener_face, cmap=plt.cm.gray)
plt.title('Wiener filter')
plt.axis('off')

plt.subplots_adjust(wspace=.05, left=.01, bottom=.01, right=.99, top=.99)

plt.show()
```

Total running time of the script: (0 minutes 0.382 seconds)

Note: Click *here* to download the full example code

6.12.16 Optimization of a two-parameter function

```
import numpy as np

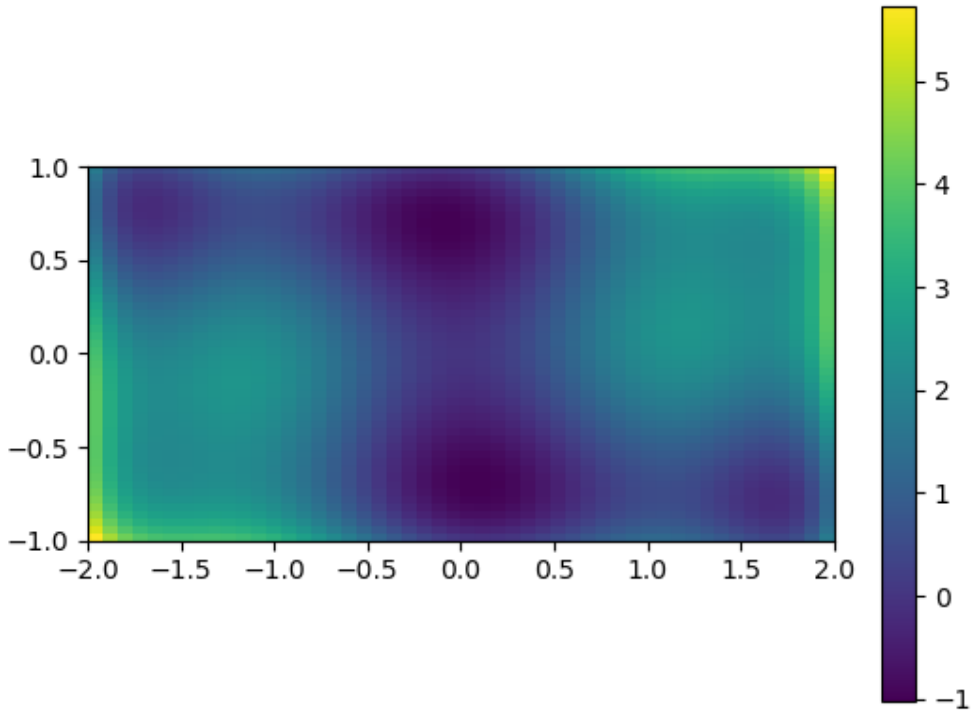

# Define the function that we are interested in
def sixhump(x):
    return ((4 - 2.1*x[0]**2 + x[0]**4 / 3.) * x[0]**2 + x[0] * x[1]
            + (-4 + 4*x[1]**2) * x[1] **2)

# Make a grid to evaluate the function (for plotting)
x = np.linspace(-2, 2)
y = np.linspace(-1, 1)
xg, yg = np.meshgrid(x, y)
```

A 2D image plot of the function

Simple visualization in 2D

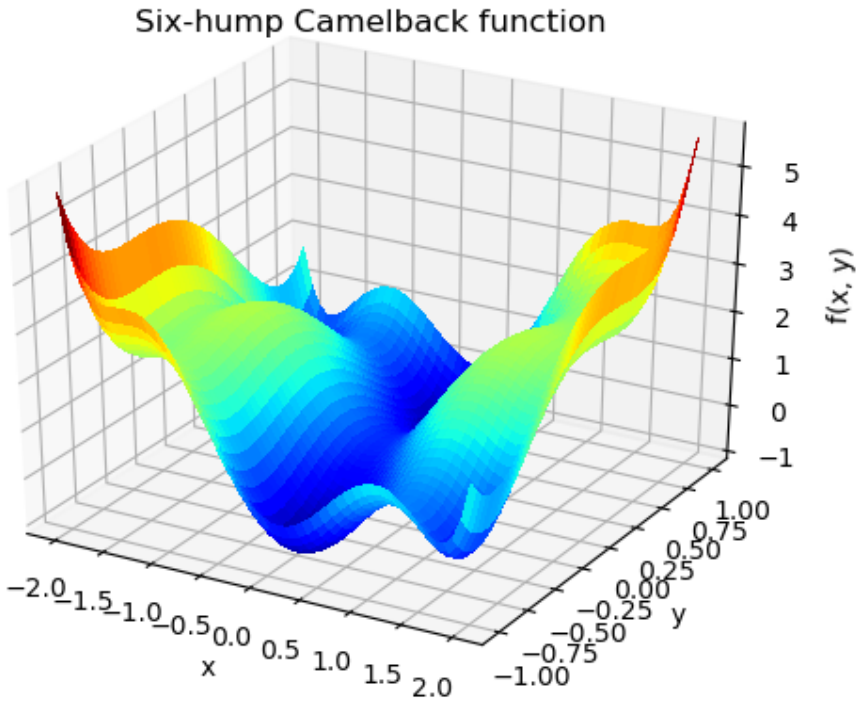
```
import matplotlib.pyplot as plt
plt.figure()
plt.imshow(sixhump([xg, yg]), extent=[-2, 2, -1, 1], origin="lower")
plt.colorbar()
```



A 3D surface plot of the function

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(xg, yg, sixhump([xg, yg]), rstride=1, cstride=1,
                       cmap=plt.cm.jet, linewidth=0, antialiased=False)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('f(x, y)')
ax.set_title('Six-hump Camelback function')
```



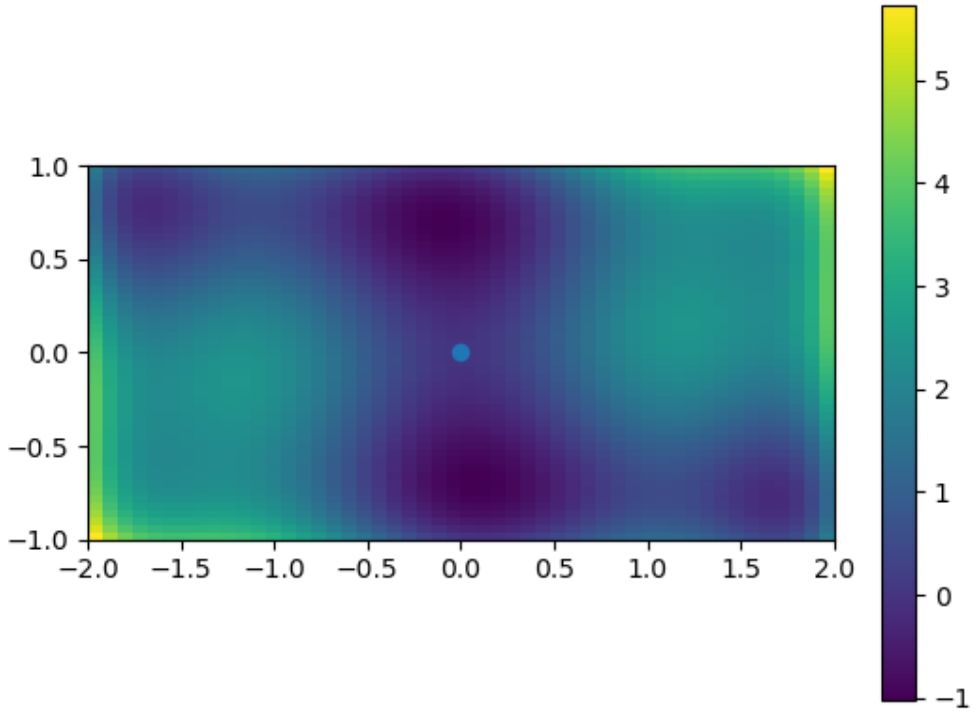
Find the minima

```
from scipy import optimize

x_min = optimize.minimize(sixhump, x0=[0, 0])

plt.figure()
# Show the function in 2D
plt.imshow(sixhump([xg, yg]), extent=[-2, 2, -1, 1], origin="lower")
plt.colorbar()
# And the minimum that we've found:
plt.scatter(x_min.x[0], x_min.x[1])

plt.show()
```



Total running time of the script: (0 minutes 0.146 seconds)

Note: Click *here* to download the full example code

6.12.17 Plotting and manipulating FFTs for filtering

Plot the power of the FFT of a signal and inverse FFT back to reconstruct a signal.

This example demonstrate `scipy.fftpack.fft()`, `scipy.fftpack.fftfreq()` and `scipy.fftpack.ifft()`. It implements a basic filter that is very suboptimal, and should not be used.

```
import numpy as np
from scipy import fftpack
from matplotlib import pyplot as plt
```

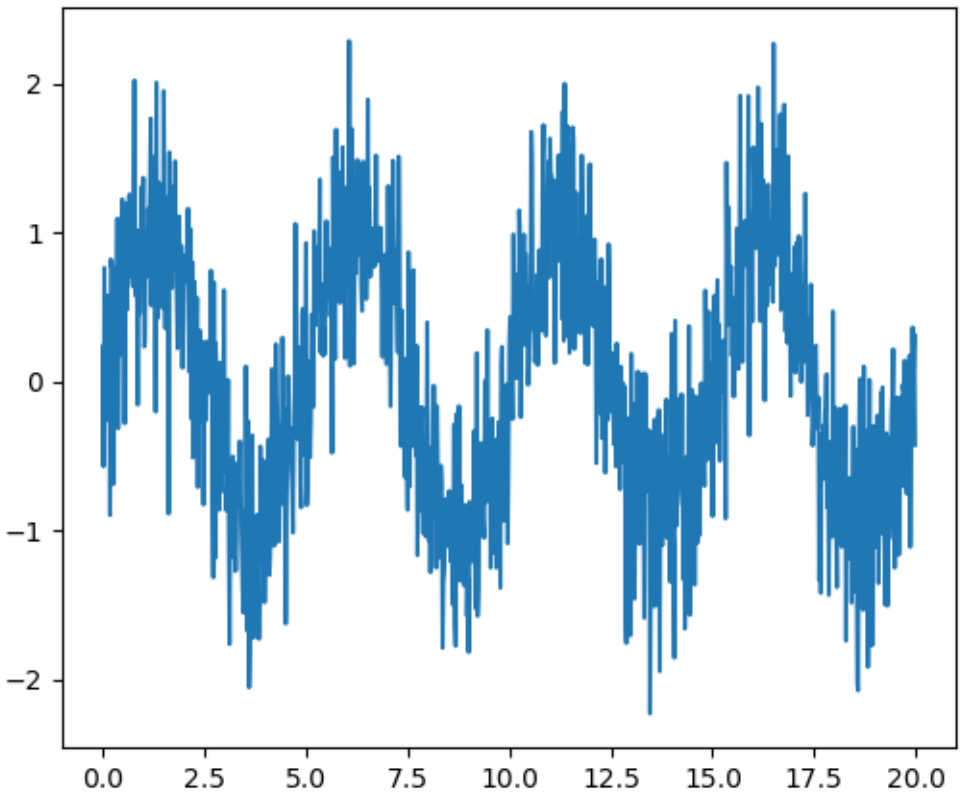
Generate the signal

```
# Seed the random number generator
np.random.seed(1234)

time_step = 0.02
period = 5.

time_vec = np.arange(0, 20, time_step)
sig = (np.sin(2 * np.pi / period * time_vec)
       + 0.5 * np.random.randn(time_vec.size))

plt.figure(figsize=(6, 5))
plt.plot(time_vec, sig, label='Original signal')
```



Compute and plot the power

```
# The FFT of the signal
sig_fft = fftpack.fft(sig)

# And the power (sig_fft is of complex dtype)
power = np.abs(sig_fft)**2

# The corresponding frequencies
sample_freq = fftpack.fftfreq(sig.size, d=time_step)

# Plot the FFT power
plt.figure(figsize=(6, 5))
plt.plot(sample_freq, power)
plt.xlabel('Frequency [Hz]')
plt.ylabel('plower')

# Find the peak frequency: we can focus on only the positive frequencies
pos_mask = np.where(sample_freq > 0)
freqs = sample_freq[pos_mask]
peak_freq = freqs[power[pos_mask].argmax()]

# Check that it does indeed correspond to the frequency that we generate
# the signal with
np.allclose(peak_freq, 1./period)

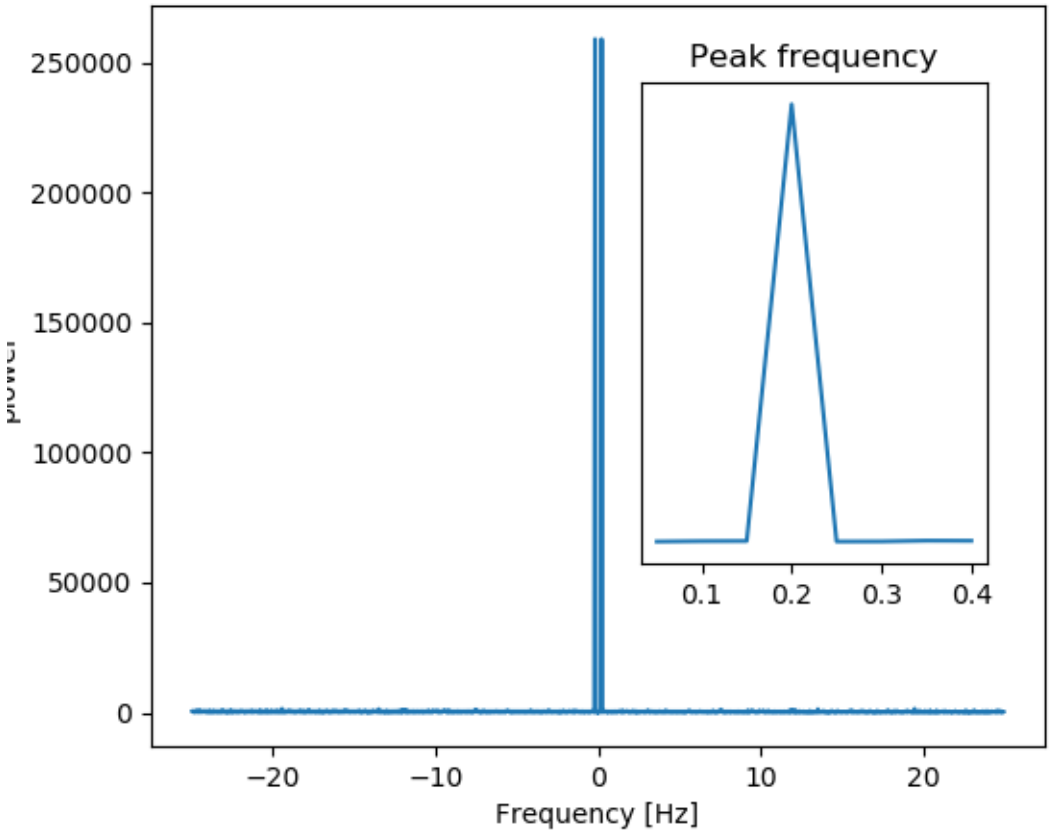
# An inner plot to show the peak frequency
axes = plt.axes([0.55, 0.3, 0.3, 0.5])
```

(continues on next page)

(continued from previous page)

```
plt.title('Peak frequency')
plt.plot(freqs[:8], power[pos_mask][:8])
plt.setp(axes, yticks=[])

# scipy.signal.find_peaks_cwt can also be used for more advanced
# peak detection
```



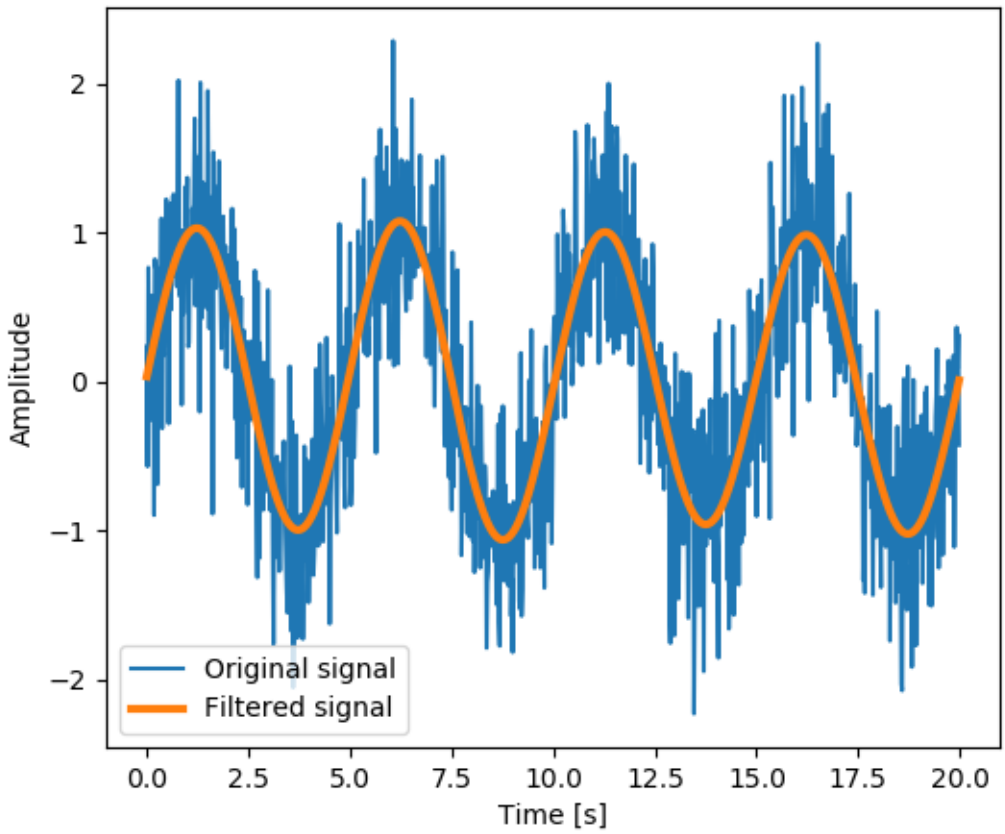
Remove all the high frequencies

We now remove all the high frequencies and transform back from frequencies to signal.

```
high_freq_fft = sig_fft.copy()
high_freq_fft[np.abs(sample_freq) > peak_freq] = 0
filtered_sig = fftpack.ifft(high_freq_fft)

plt.figure(figsize=(6, 5))
plt.plot(time_vec, sig, label='Original signal')
plt.plot(time_vec, filtered_sig, linewidth=3, label='Filtered signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')

plt.legend(loc='best')
```



Note This is actually a bad way of creating a filter: such brutal cut-off in frequency space does not control distorsion on the signal.

Filters should be created using the scipy filter design code

```
plt.show()
```

Total running time of the script: (0 minutes 0.110 seconds)

6.12.18 Solutions of the exercises for scipy

Note: Click *here* to download the full example code

Crude periodicity finding

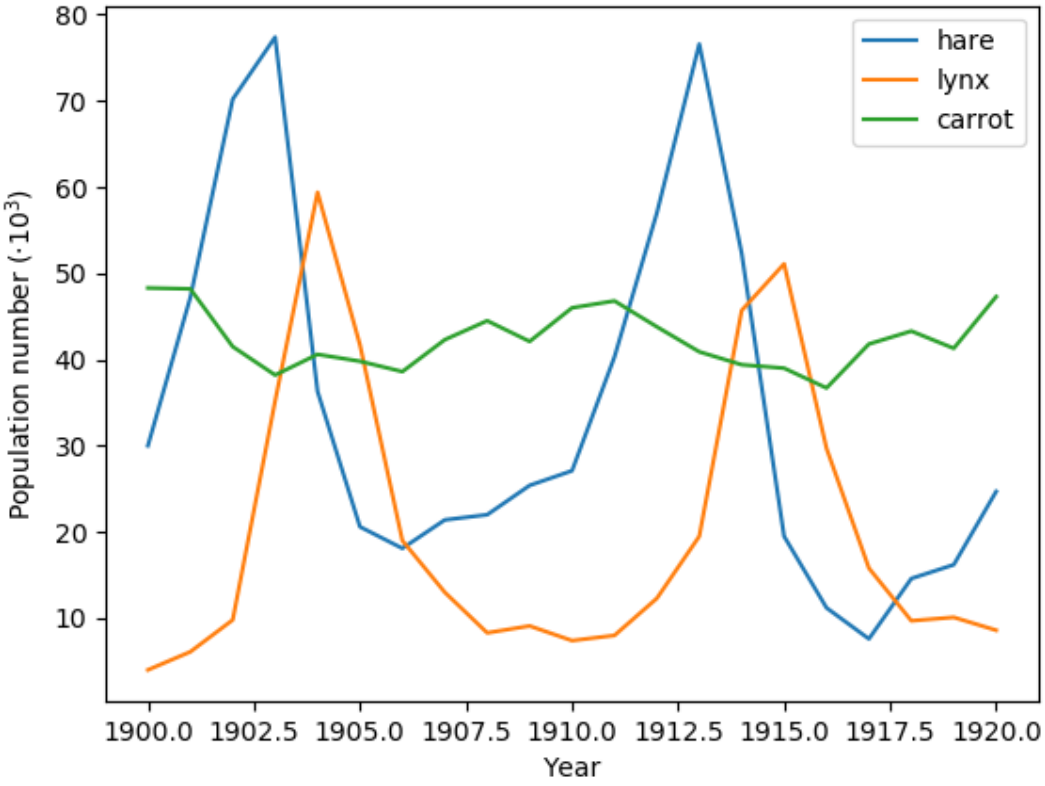
Discover the periods in evolution of animal populations (`../../../../data/populations.txt`)

Load the data

```
import numpy as np
data = np.loadtxt('../../../../data/populations.txt')
years = data[:, 0]
populations = data[:, 1:]
```

Plot the data

```
import matplotlib.pyplot as plt
plt.figure()
plt.plot(years, populations * 1e-3)
plt.xlabel('Year')
plt.ylabel('Population number ($\cdot10^3$)')
plt.legend(['hare', 'lynx', 'carrot'], loc=1)
```



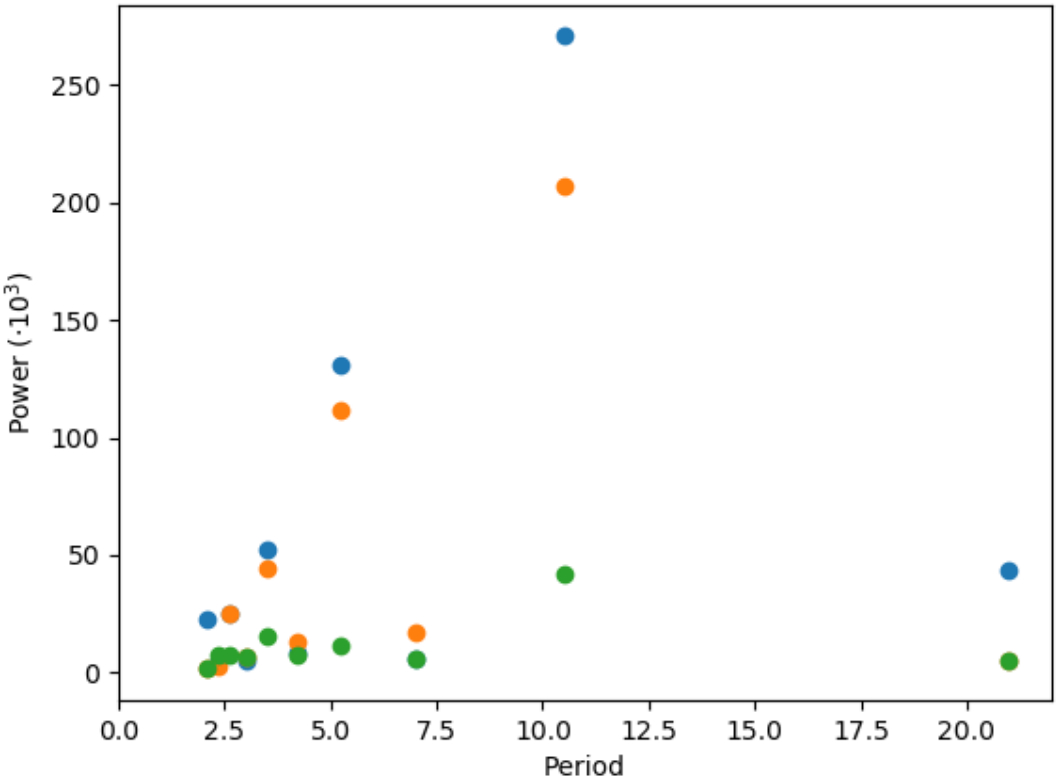
Plot its periods

```
from scipy import fftpack

ft_populations = fftpack.fft(populations, axis=0)
frequencies = fftpack.fftfreq(populations.shape[0], years[1] - years[0])
periods = 1 / frequencies

plt.figure()
plt.plot(periods, abs(ft_populations) * 1e-3, 'o')
plt.xlim(0, 22)
plt.xlabel('Period')
plt.ylabel('Power ($\cdot10^3$)')

plt.show()
```



There's probably a period of around 10 years (obvious from the plot), but for this crude a method, there's not enough data to say much more.

Total running time of the script: (0 minutes 0.027 seconds)

Note: Click *here* to download the full example code

Curve fitting: temperature as a function of month of the year

We have the min and max temperatures in Alaska for each months of the year. We would like to find a function to describe this yearly evolution.

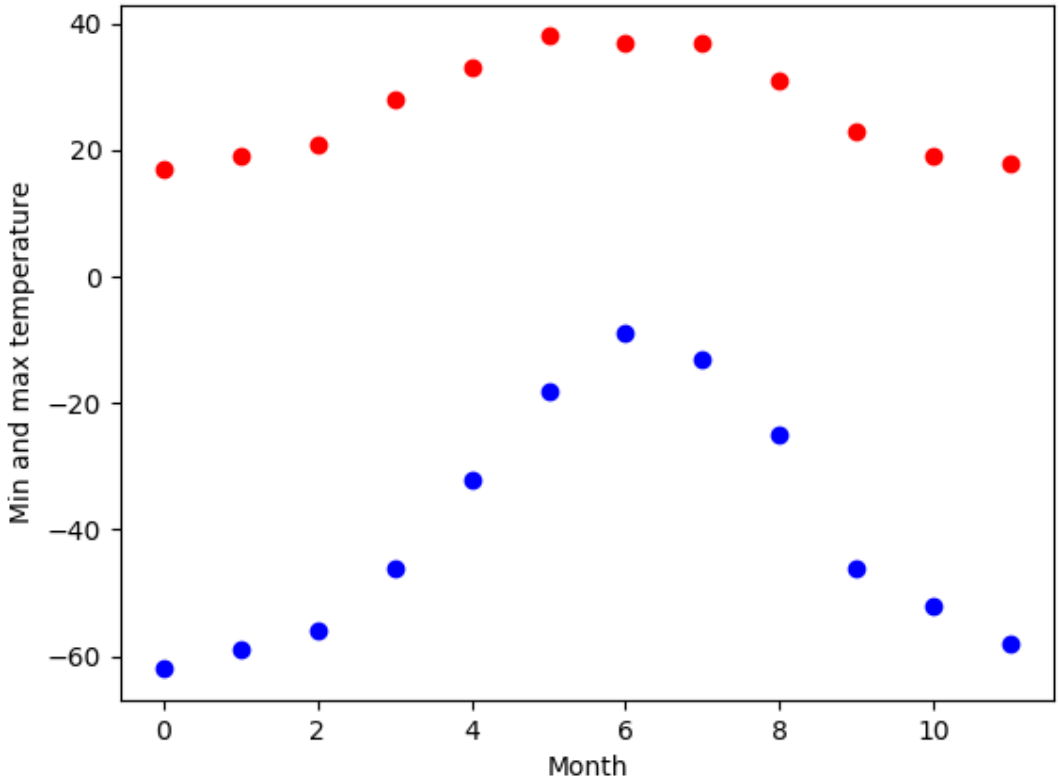
For this, we will fit a periodic function.

The data

```
import numpy as np

temp_max = np.array([17,  19,  21,  28,  33,  38, 37,  37,  31,  23,  19,  18])
temp_min = np.array([-62, -59, -56, -46, -32, -18, -9, -13, -25, -46, -52, -58])

import matplotlib.pyplot as plt
months = np.arange(12)
plt.plot(months, temp_max, 'ro')
plt.plot(months, temp_min, 'bo')
plt.xlabel('Month')
plt.ylabel('Min and max temperature')
```



Fitting it to a periodic function

```
from scipy import optimize
def yearly_temps(times, avg, ampl, time_offset):
    return (avg
            + ampl * np.cos((times + time_offset) * 2 * np.pi / times.max()))

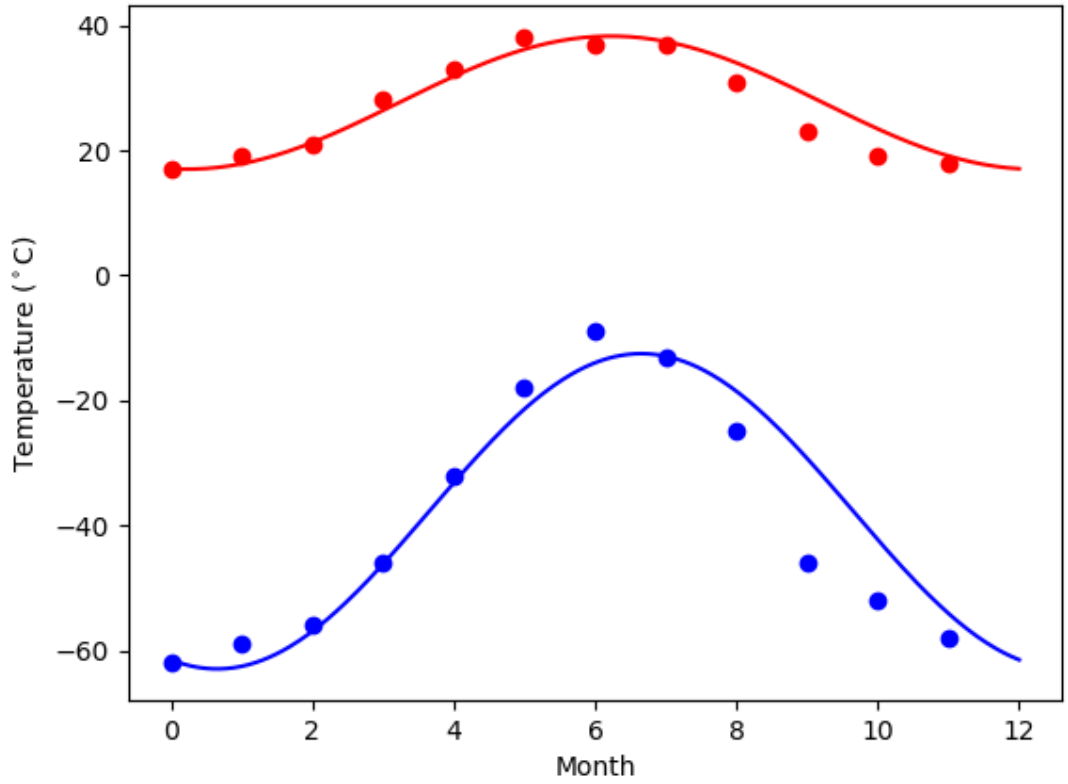
res_max, cov_max = optimize.curve_fit(yearly_temps, months,
                                      temp_max, [20, 10, 0])
res_min, cov_min = optimize.curve_fit(yearly_temps, months,
                                      temp_min, [-40, 20, 0])
```

Plotting the fit

```
days = np.linspace(0, 12, num=365)

plt.figure()
plt.plot(months, temp_max, 'ro')
plt.plot(days, yearly_temps(days, *res_max), 'r-')
plt.plot(months, temp_min, 'bo')
plt.plot(days, yearly_temps(days, *res_min), 'b-')
plt.xlabel('Month')
plt.ylabel('Temperature ($^\circ$C)')

plt.show()
```



Total running time of the script: (0 minutes 0.027 seconds)

Note: Click *here* to download the full example code

Simple image blur by convolution with a Gaussian kernel

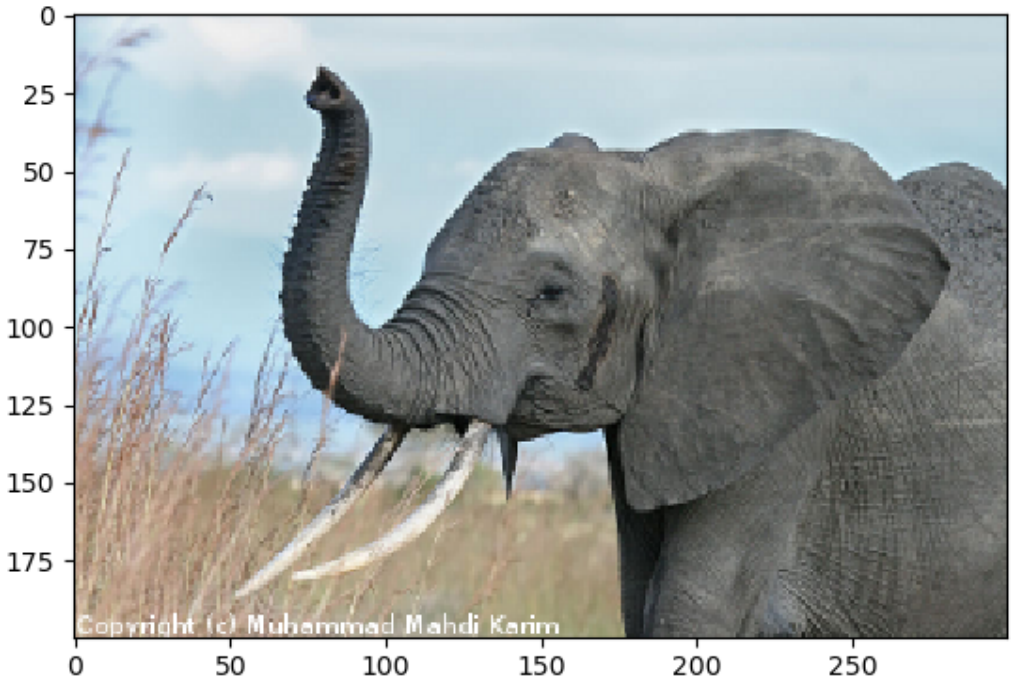
Blur an an image (`../../../../data/elephant.png`) using a Gaussian kernel.

Convolution is easy to perform with FFT: convolving two signals boils down to multiplying their FFTs (and performing an inverse FFT).

```
import numpy as np
from scipy import fftpack
import matplotlib.pyplot as plt
```

The original image

```
# read image
img = plt.imread('../../../../data/elephant.png')
plt.figure()
plt.imshow(img)
```



Prepare an Gaussian convolution kernel

```
# First a 1-D  Gaussian
t = np.linspace(-10, 10, 30)
bump = np.exp(-0.1*t**2)
bump /= np.trapz(bump) # normalize the integral to 1

# make a 2-D kernel out of it
kernel = bump[:, np.newaxis] * bump[np.newaxis, :]
```

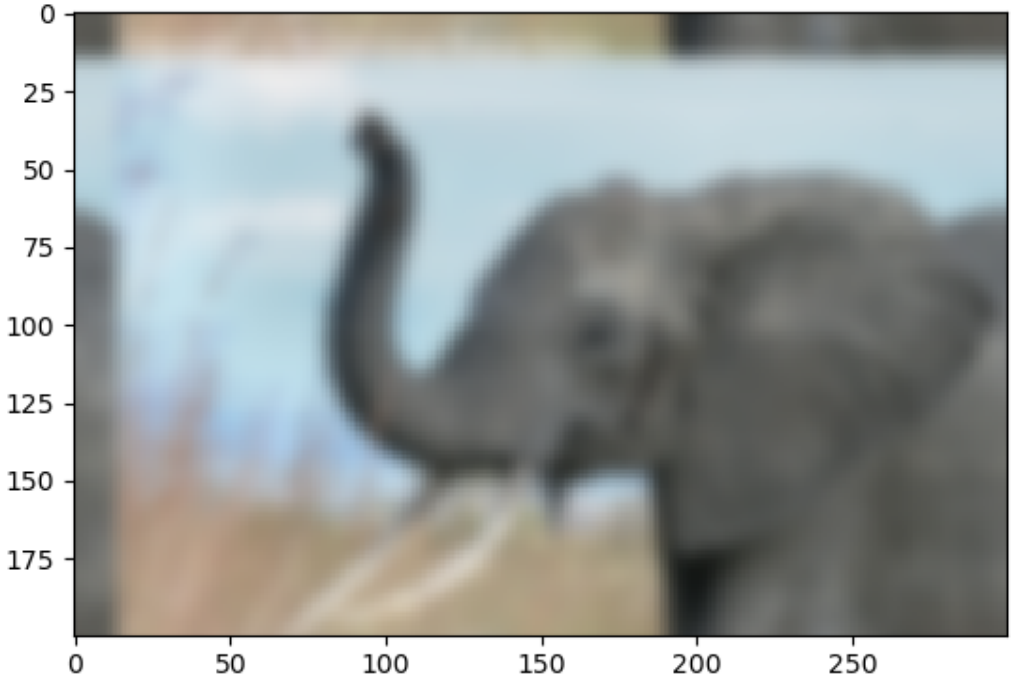
Implement convolution via FFT

```
# Padded fourier transform, with the same shape as the image
# We use :func:`scipy.signal.fftpack.fft2` to have a 2D FFT
kernel_ft = fftpack.fft2(kernel, shape=img.shape[:2], axes=(0, 1))

# convolve
img_ft = fftpack.fft2(img, axes=(0, 1))
# the 'newaxis' is to match to color direction
img2_ft = kernel_ft[:, :, np.newaxis] * img_ft
img2 = fftpack.ifft2(img2_ft, axes=(0, 1)).real

# clip values to range
img2 = np.clip(img2, 0, 1)

# plot output
plt.figure()
plt.imshow(img2)
```



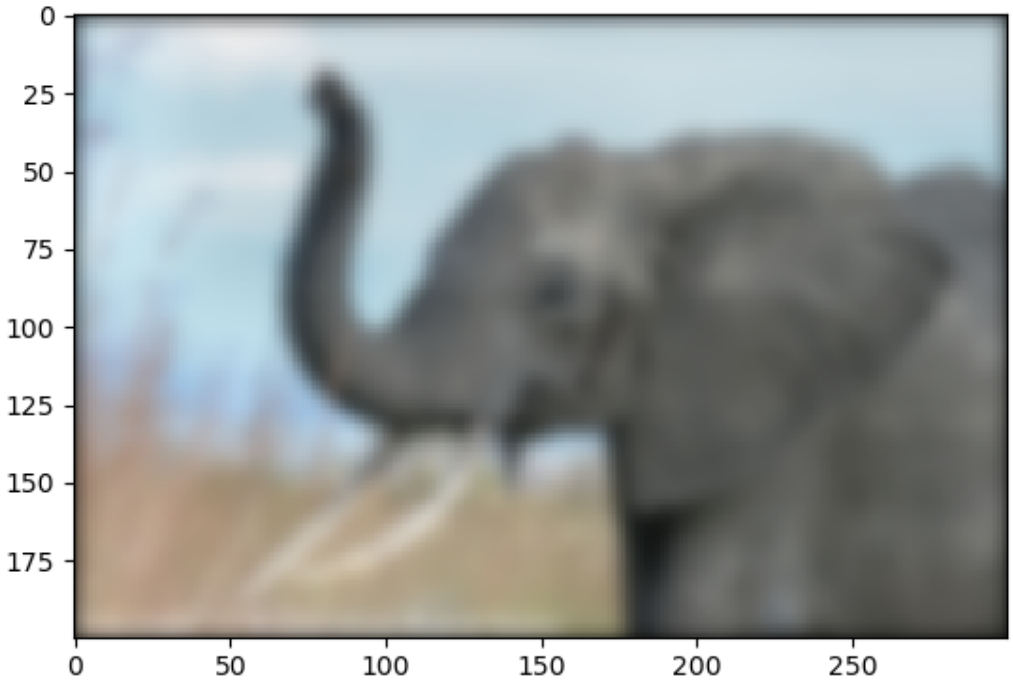
Further exercise (only if you are familiar with this stuff):

A "wrapped border" appears in the upper left and top edges of the image. This is because the padding is not done correctly, and does not take the kernel size into account (so the convolution "flows out of bounds of the image"). Try to remove this artifact.

A function to do it: `scipy.signal.fftconvolve()`

The above exercise was only for didactic reasons: there exists a function in scipy that will do this for us, and probably do a better job: `scipy.signal.fftconvolve()`

```
from scipy import signal
# mode='same' is there to enforce the same output shape as input arrays
# (ie avoid border effects)
img3 = signal.fftconvolve(img, kernel[:, :, np.newaxis], mode='same')
plt.figure()
plt.imshow(img3)
```



Note that we still have a decay to zero at the border of the image. Using `scipy.ndimage.gaussian_filter()` would get rid of this artifact

```
plt.show()
```

Total running time of the script: (0 minutes 0.064 seconds)

Note: Click *here* to download the full example code

Image denoising by FFT

Denoise an image (`../../../../data/moonlanding.png`) by implementing a blur with an FFT.

Implements, via FFT, the following convolution:

$$f_1(t) = \int dt'\, K(t-t') f_0(t')$$

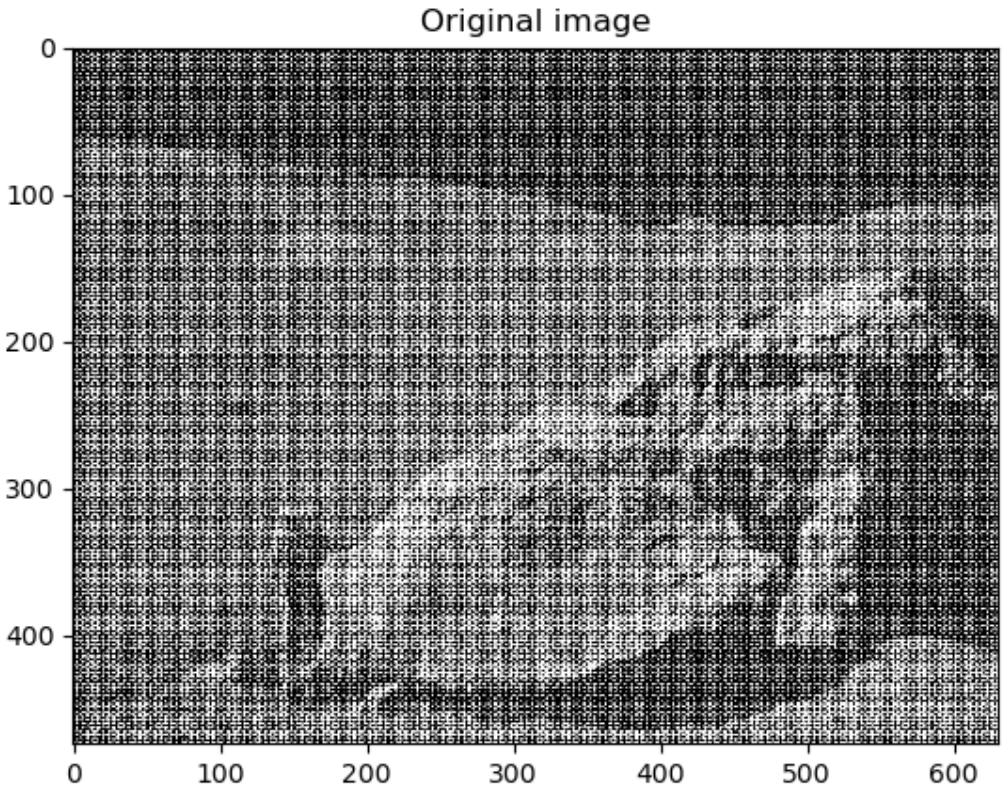
$$\tilde{f}_1(\omega) = \tilde{K}(\omega)\tilde{f}_0(\omega)$$

Read and plot the image

```
import numpy as np
import matplotlib.pyplot as plt

im = plt.imread('../../../../data/moonlanding.png').astype(float)

plt.figure()
plt.imshow(im, plt.cm.gray)
plt.title('Original image')
```



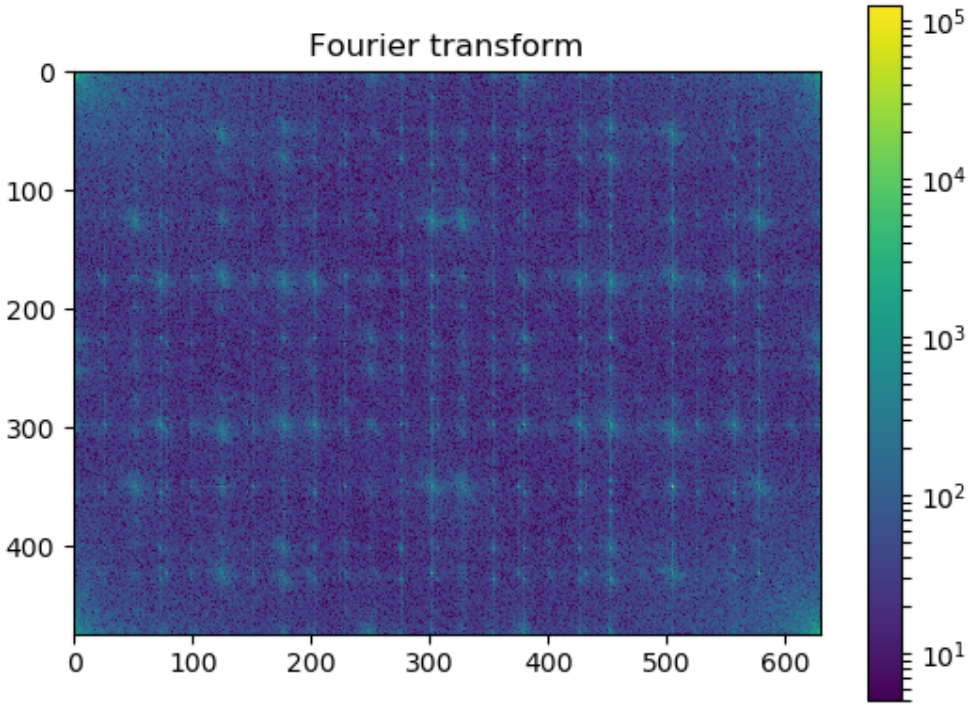
Compute the 2d FFT of the input image

```
from scipy import fftpack
im_fft = fftpack.fft2(im)

# Show the results

def plot_spectrum(im_fft):
    from matplotlib.colors import LogNorm
    # A logarithmic colormap
    plt.imshow(np.abs(im_fft), norm=LogNorm(vmin=5))
    plt.colorbar()

plt.figure()
plot_spectrum(im_fft)
plt.title('Fourier transform')
```



Filter in FFT

```
# In the lines following, we'll make a copy of the original spectrum and
# truncate coefficients.

# Define the fraction of coefficients (in each direction) we keep
keep_fraction = 0.1

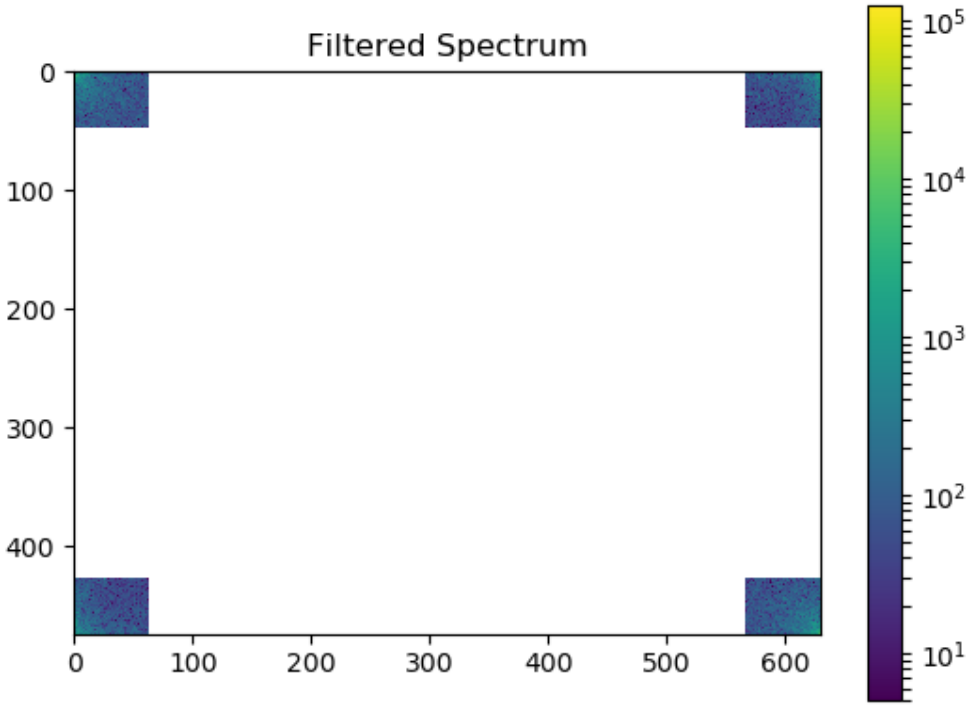
# Call ff a copy of the original transform. Numpy arrays have a copy
# method for this purpose.
im_fft2 = im_fft.copy()

# Set r and c to be the number of rows and columns of the array.
r, c = im_fft2.shape

# Set to zero all rows with indices between r*keep_fraction and
# r*(1-keep_fraction):
im_fft2[int(r*keep_fraction):int(r*(1-keep_fraction))] = 0

# Similarly with the columns:
im_fft2[:, int(c*keep_fraction):int(c*(1-keep_fraction))] = 0

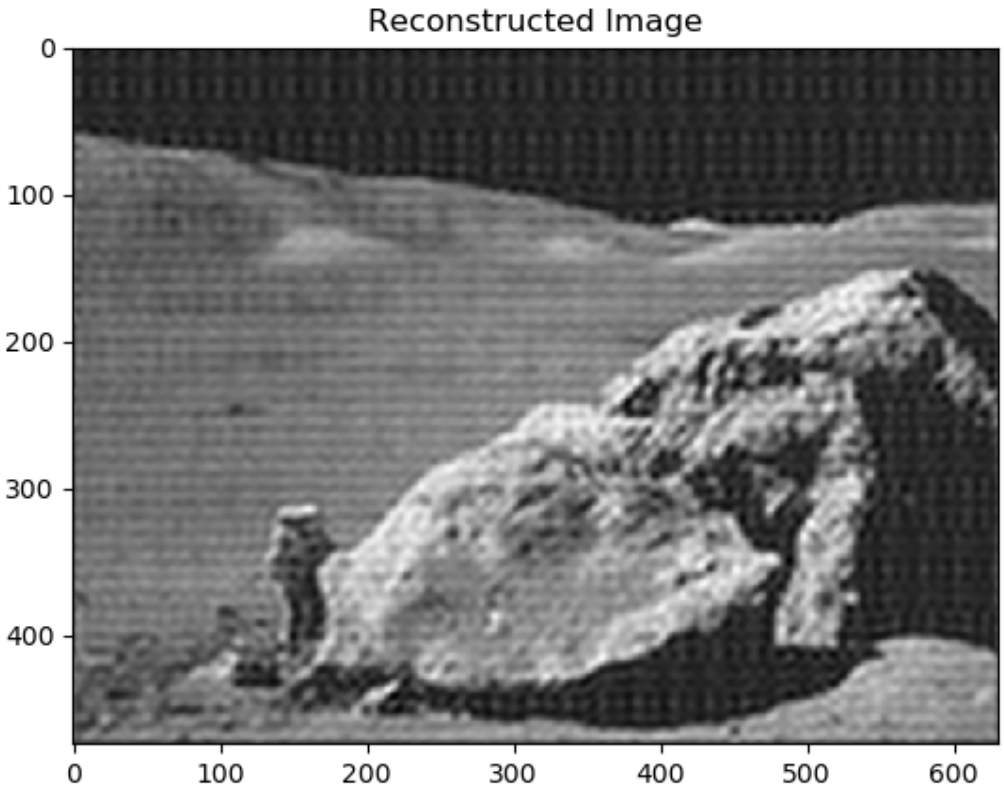
plt.figure()
plot_spectrum(im_fft2)
plt.title('Filtered Spectrum')
```



Reconstruct the final image

```
# Reconstruct the denoised image from the filtered spectrum, keep only the
# real part for display.
im_new = fftpack.ifft2(im_fft2).real

plt.figure()
plt.imshow(im_new, plt.cm.gray)
plt.title('Reconstructed Image')
```



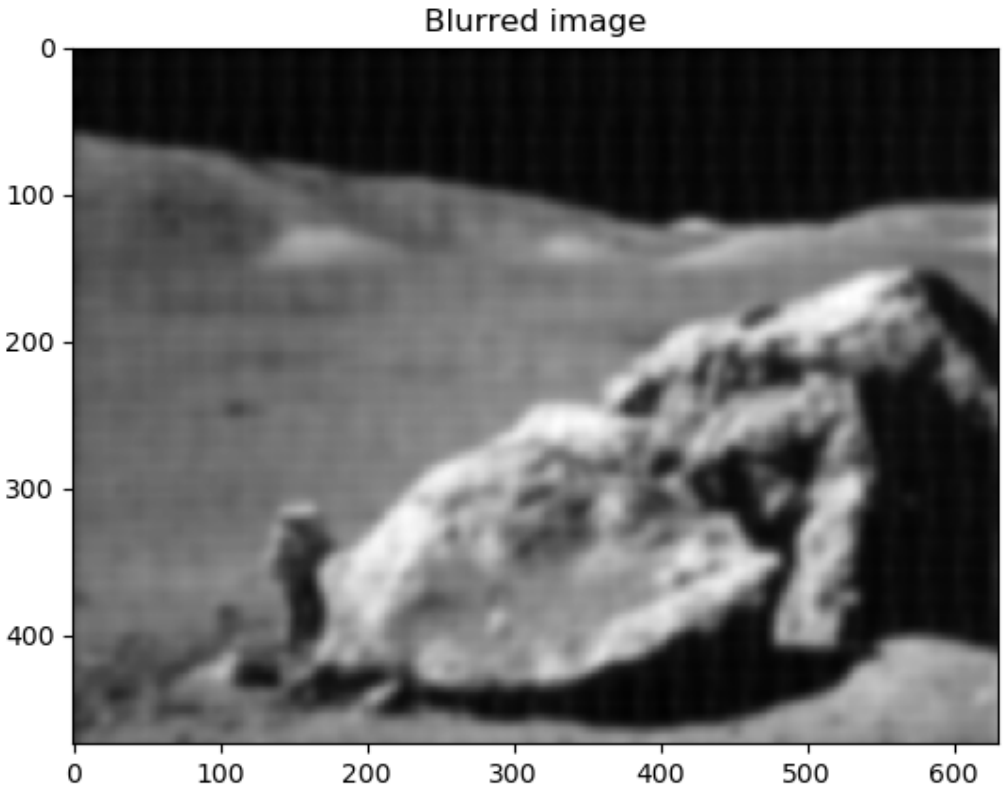
Easier and better: `scipy.ndimage.gaussian_filter()`

Implementing filtering directly with FFTs is tricky and time consuming. We can use the Gaussian filter from `scipy.ndimage`

```
from scipy import ndimage
im_blur = ndimage.gaussian_filter(im, 4)

plt.figure()
plt.imshow(im_blur, plt.cm.gray)
plt.title('Blurred image')

plt.show()
```



Total running time of the script: (0 minutes 0.211 seconds)

See also:

References to go further

- Some chapters of the advanced and the packages and applications parts of the scipy lectures
- The scipy cookbook